CINESCOPE

Movie Review Analysis and Recommendation

A small runnable mid-sem project and a beginner-to-expert handbook

RUNNABLE CORE	STANDARD-LIBRARY PYTHON + HTML/CSS/JS
BIG-DATA LAYER	SCALA 2.12 + SPARK 3.5.7 BATCH JOB
OPTIONAL NLP	TF-IDF, N-GRAM LM, WORD2VEC, DISTILBERT, ASPECTS, LDA
EDITION	28 SEPTEMBER 2026

Scope note

The application is intentionally small. Advanced chapters describe executable optional experiments and valid evaluation protocols. Unrun training and unmeasured research results are identified explicitly; no metric is fabricated.

Contents

1. Read this first: two different deliverables 12

Three levels of completion . . . 12

How to study . . . 12

2. Your problem statement and mid-sem objective 12

What the demonstrator should prove . . . 13

What success does not require yet . . . 13

A sensible project title . . . 13

Mid-sem deliverable checklist . . . 13

3. Basic vocabulary without prerequisites 13

The three kinds of text task . . . 13

Numbers that sound similar . . . 13

Four software layers . . . 14

4. The architecture: online and offline paths 14

Why not run Spark for every keystroke? . . . 14

Where integration really occurs . . . 14

5. Install and run the smallest version 14

Your first five interactions . . . 15

Common startup mistakes . . . 15

What gets saved? . . . 15

6. Understanding the frontend 15

What "real time" means here . . . 15

Displaying text safely and clearly . . . 15

Search and history . . . 16

A beginner debugging exercise . . . 16

7. Backend and API contracts 16

Example analysis request . . . 16

Input boundaries . . . 16

How errors appear . . . 16

Minimal persistence choice . . . 16

8. Dataset roles: MovieLens is not the NLP corpus 17

What the new package contains . . . 17

Three identities to keep separate . . . 17

A valid data statement . . . 17

9. IDs, joins and trustworthy data integration 17

Example of a valid local join . . . 17
The dangerous shortcut . . . 18
Provenance fields . . . 18

10. Text preprocessing, step by step **18**

Why keep negation? . . . 18
Tokenization . . . 18
Spelling and slang policy . . . 19
Useful manual test set . . . 19

11. Sentences, clauses and duplicate handling **19**

Why splitting helps . . . 19
Exact duplicates versus near duplicates . . . 19
What the small app does . . . 19
Research exercise . . . 19

12. Morphology: stemming versus lemmatization **20**

Why context matters . . . 20
A controlled project experiment . . . 20
Expected trade-offs, not promised results . . . 20

13. Overall rule-based sentiment: the exact calculation **20**

Worked example . . . 21
Strengths . . . 21
Weaknesses . . . 21
Honest reporting . . . 21

14. Negation, intensity and mixed feelings **21**

Examples to trace . . . 21
Intensifiers are not learned here . . . 21
Balanced score versus neutral meaning . . . 22
An extension worth evaluating . . . 22

15. Six aspects: from words to local evidence **22**

Worked trace . . . 22
Important limitations . . . 22
Evaluation target . . . 22

16. Theme and genre scores: what they really mean **23**

What a detected tag contains . . . 23
A subtle limitation . . . 23
How to improve later . . . 23

17. Syntax and semantics: beyond nearby words **23**

A practical extension . . . 23

Why this is still imperfect . . . 24
The right comparison . . . 24
Current implementation boundary . . . 24

18. One-hot vectors, BoW and TF-IDF 24

TF-IDF intuition . . . 24
Small worked example . . . 24
Why fit only on training data? . . . 24
What the experiment controls . . . 25

19. Classification n-grams are not language models 25

Two different questions . . . 25
Example . . . 25
Project mapping . . . 25

20. Logistic regression from first principles 25

Worked numerical example . . . 26
Why it fits mid-sem work . . . 26
Regularization . . . 26
Inspecting errors and weights . . . 26

21. Preparing IMDb without leaking labels 26

What the script writes . . . 26
Duplicate and split policy . . . 26
What to record . . . 27
Avoid silent subsampling . . . 27

22. A controlled BoW versus TF-IDF experiment 27

Procedure . . . 27
Reporting table to fill after execution . . . 27
What could change the conclusion? . . . 27

23. Deploying a trained baseline without a heavy server 28

Why export JSON? . . . 28
Consistency requirement . . . 28
How the interface changes . . . 28
Practical sequence . . . 28
Evaluation boundary . . . 28

24. A genuine bigram and trigram language model 28

Vocabulary and boundaries . . . 29
Worked example . . . 29
Why this lab is separate . . . 29
Limitations to explain . . . 29

25. Perplexity and next-word prediction 29
Numerical example . . . 29
What the script produces . . . 29
A common surprising result . . . 29
Suggested extension . . . 30
26. Collocations: frequency and PMI 30
Worked intuition . . . 30
What the supplied experiment does . . . 30
How to improve it . . . 30
What to report . . . 30
27. Word2Vec: CBOW and Skip-gram 30
Project experiment . . . 31
Cosine similarity . . . 31
A useful report . . . 31
Practical limit . . . 31
28. GloVe comparison and embedding experiments 31
Keep the comparison interpretable . . . 31
Quantitative options . . . 31
Joining embeddings to recommendation . . . 32
Current project boundary . . . 32
29. DistilBERT training and fair comparison 32
Commands and resources . . . 32
What is held constant . . . 32
Training-loop intuition . . . 32
A better final comparison . . . 32
30. Human aspect annotations: the missing foundation 33
Label definitions . . . 33
Annotation process . . . 33
Sample-size honesty . . . 33
Evidence spans . . . 33
31. Training and evaluating six aspect classifiers 33
Outputs . . . 34
Why per-aspect metrics matter . . . 34
A possible stronger architecture . . . 34
Avoid an evaluation shortcut . . . 34
Serving integration decision . . . 34
32. Topic modelling with LDA 34
Topic versus theme . . . 34

Evaluation . . . 34
A worked analysis workflow . . . 35
Recommendation extension . . . 35

33. Movie-specific named entity recognition **35**

Two stages . . . 35
A small extension design . . . 35
Evaluation data . . . 35
Example failure . . . 35
Current scope . . . 35

34. Review summarization with supporting evidence **36**

A simple final-stage pipeline . . . 36
Evaluation rubric . . . 36
Current scope . . . 36

35. Conversational QA: retrieve before answering **36**

A minimal grounded design . . . 36
Retrieval and generation are separate . . . 37
Evaluation set . . . 37
Conversation state . . . 37
Mid-sem boundary . . . 37

36. Recommendations: what history represents **37**

Signed rating weight . . . 37
Genre preference . . . 37
Example . . . 38
What this method cannot infer . . . 38

37. The exact hybrid ranking formula **38**

Theme component . . . 38
Aspect component . . . 38
Rating prior . . . 38
Example calculation . . . 38

38. Connecting real reviews to recommendation features **39**

Required data contract . . . 39
Four extension options . . . 39
Missing information . . . 39
Temporal leakage . . . 39
Ablation study . . . 39

39. Cold start, fallback and recommendation limits **39**

What the prototype does . . . 40
Why recommendations can look repetitive . . . 40

Popularity and selection bias . . . 40
Minimum honest explanation . . . 40

40. Evaluating recommendations properly **40**

Core ranking metrics . . . 40
Tiny example . . . 40
Baselines and coverage . . . 41
Current utility . . . 41

41. Big data concepts before Spark syntax **41**

MovieLens operations that scale naturally . . . 41
The hidden cost: shuffles . . . 41
Skew . . . 41
What to claim in mid-sem . . . 41

42. Scala essentials for this project **41**

Values and chaining . . . 42
Columns are expressions . . . 42
Why types matter . . . 42
Build files . . . 42
Viva explanation . . . 42

43. Walk through MovieStats.scala **42**

Read and clean . . . 42
Aggregate and join . . . 42
Genre aggregation . . . 43
Write outputs . . . 43

44. Build, run and connect the Scala output **43**

Output interpretation . . . 43
Troubleshooting . . . 43
Validation status . . . 43

45. From local batch to a genuine scale experiment **44**

Benchmark design . . . 44
Scaling quantities . . . 44
Storage choices . . . 44
What to inspect in Spark . . . 44
A result worth presenting . . . 44

46. ALS and streaming: keep them as final-stage extensions **44**

ALS intuition . . . 45
Evaluation boundary . . . 45
Streaming intuition . . . 45
Why defer these features? . . . 45

47. Classification metrics and confusion matrices 45

Worked example . . . 45

Why accuracy can mislead . . . 46

Aspect metrics . . . 46

Evidence quality . . . 46

48. Evaluating the complete NLP pipeline 46

Input examples: schema illustrations, not benchmark results . . . 46

Important limits . . . 46

End-to-end assessment . . . 46

49. Calibration, uncertainty and error analysis 47

A simple diagnostic . . . 47

Uncertainty is broader than one number . . . 47

Error buckets . . . 47

Paired comparisons . . . 47

What to improve first . . . 47

50. Reproducibility and reporting actual results 47

Save a run record . . . 47

Training versus testing status . . . 48

Environment files . . . 48

Honest tables . . . 48

A useful final conclusion . . . 48

51. Advanced foundations: loss, attention and representation 48

Logistic loss . . . 48

Attention intuition . . . 48

Representation versus objective . . . 49

Expert habit . . . 49

52. The full 15-topic syllabus map 49

How to use this table . . . 49

53. The six priorities as an execution plan 50

Priority 1: Word2Vec and GloVe . . . 50

Priority 2: a genuine n-gram model . . . 50

Priority 3: train DistilBERT . . . 50

Priority 4: train aspects . . . 50

Priority 5: use review information in ranking . . . 50

Priority 6: evaluate the whole pipeline . . . 50

Dependency order . . . 50

54. A practical semester schedule 50

Mid-sem stopping point . . . 51
When to simplify further . . . 51
When to expand . . . 51

55. A five-minute mid-sem demonstration 51

Minute 1: live review analysis . . . 51
Minute 2: missing evidence and negation . . . 52
Minute 3: history and recommendation . . . 52
Minute 4: data and Scala . . . 52
Minute 5: limitations and next stage . . . 52
A suitable spoken conclusion . . . 52

56. Viva bank: beginner and implementation questions 52

1. What problem does CineScope solve? . . . 52
2. What is sentiment analysis? . . . 52
3. What is an aspect? . . . 52
4. Why do some scores show no evidence? . . . 52
5. What does 100 mean in the default app? . . . 53
6. How does live analysis work? . . . 53
7. Why Python for the backend? . . . 53
8. Why not use only MovieLens for NLP? . . . 53
9. Is the demo catalog real? . . . 53
10. How is history stored? . . . 53
11. How does the engine handle mixed opinions? . . . 53
12. How is negation handled? . . . 53
13. Do you remove all stop words? . . . 53
14. What is the difference between a theme and genre? . . . 53
15. What does theme coverage mean? . . . 53
16. Why exclude seen films? . . . 53
17. How do dislikes affect recommendations? . . . 54
18. Is this collaborative filtering? . . . 54
19. What is the main limitation? . . . 54
20. What did you simplify from the final project? . . . 54

57. Viva bank: training, big data and research questions 54

21. What is BoW? . . . 54
22. How does TF-IDF differ? . . . 54
23. Why logistic regression? . . . 54
24. What is data leakage? . . . 54
25. Why a validation set? . . . 54
26. Why not report only accuracy? . . . 54
27. Why compare models on identical test IDs? . . . 54
28. Is a bigram TF-IDF feature a language model? . . . 55

29. Why smoothing in an n-gram model? . . . 55
30. Can a trigram model be worse than a bigram? . . . 55
31. What is CBOW versus Skip-gram? . . . 55
32. Why can good and bad be near in word-vector space? . . . 55
33. What does DistilBERT add? . . . 55
34. Why can't overall labels train all aspects? . . . 55
35. How will you evaluate NER? . . . 55
36. How will you evaluate summaries? . . . 55
37. What is a Spark DataFrame? . . . 55
38. What is a shuffle? . . . 55
39. Why avoid collecting all data? . . . 55
40. Why coalesce the small UI export? . . . 55
41. Does local[2] prove cluster scalability? . . . 56
42. Why can Spark be slower on small data? . . . 56
43. Why does low ALS RMSE not guarantee good recommendations? . . . 56
44. What is NDCG? . . . 56
45. What is cold start? . . . 56
46. How will you prove review features help? . . . 56
47. What is calibration? . . . 56
48. Which advanced results are already measured? . . . 56
49. What should final-stage work prioritize? . . . 56
50. What makes the project scientifically credible? . . . 56

58. Troubleshooting and common misconceptions 57

Misconception: a high score proves the model is correct . . . 57
Misconception: more advanced always means better . . . 57
Misconception: all unknowns should become neutral . . . 57
Misconception: adding an evaluation script completes evaluation . . . 57

59. Code reading guide and ownership of each file 57

How to explain a function . . . 58
Source appendix . . . 58

60. References, attribution and final research checklist 58

R1. MovieLens-small README and GroupLens . . . 58
R2. Stanford Large Movie Review Dataset . . . 58
R3. scikit-learn TF-IDF documentation . . . 59
R4. Gensim Word2Vec documentation . . . 59
R5. Stanford GloVe project . . . 59
R6. Hugging Face Transformers model APIs . . . 59
R7. Apache Spark 3.5.7 quick start . . . 59
Final research checklist . . . 59

Appendix A. Complete selected source listings 60

How to read this appendix . . . 60
app.py . . . 61
engine.py . . . 63
import_movielens.py . . . 65
web/index.html . . . 65
web/app.js . . . 66
web/style.css . . . 67
scala/src/main/scala/MovieStats.scala . . . 67
scala/import_spark_catalog.py . . . 68
optional/prepare_imdb.py . . . 68
optional/train_baselines.py . . . 69
optional/ngram_lab.py . . . 70
optional/word_vectors.py . . . 70
optional/train_distilbert.py . . . 71
optional/train_aspects.py . . . 72
optional/corpus_lab.py . . . 72
optional/evaluate_pipeline.py . . . 73
tests/test_core.py . . . 74

1. Read this first: two different deliverables

This handbook accompanies CineScope Mid-sem, a deliberately small version of your final movie project. The runnable app is short enough to explain in a viva. The handbook is much broader: it takes you from basic terminology to the experiments needed for a stronger final submission. Do not confuse an explanation of an advanced method with a claim that the method has already been trained.

The core demo runs with Python's standard library. It has an HTML/CSS/JavaScript frontend, a small Python HTTP server, a transparent text-analysis engine and a tiny fictional catalog. Scala is used in one separate Spark batch program. This division keeps the live app easy to run while preserving a genuine big-data programming component.

Three levels of completion

- Working core: live review analysis, six aspects, explicit theme/genre mentions, browser history, explainable recommendations and a MovieLens importer.
- Optional executable experiments: BoW versus TF-IDF, bigram/trigram language models, CBOW versus Skip-gram, GloVe comparison, DistilBERT training, six aspect classifiers, PMI and LDA. These require your data and additional packages.
- Advanced design and evaluation: dependency-based extraction, movie-specific NER, summaries, retrieval QA, ALS, large-scale benchmarking and stronger research methodology. These are taught here as extensions; they are not hidden features of the small app.

How to study

First run the app and type one review. Then read chapters 2-16 to understand every visible score. Next study the recommendation and Scala chapters. Finally work through the training and evaluation chapters with real data. Use the source appendix while tracing variables, and the viva bank to practise concise answers.

No measured accuracy from the uploaded final project is transferred to this new prototype. Its packaged model files are intentionally not reused. The synthetic catalog is for demonstrating behavior, not evaluating NLP quality. Missing evidence is represented explicitly instead of inventing an opinion score.

2. Your problem statement and mid-sem objective

A single movie rating hides several different opinions. A person may enjoy the acting and music while disliking the ending. Another person may care much more about visuals than story. CineScope turns a written review into an understandable breakdown and uses viewing preferences to suggest other films.

A suitable mid-sem problem statement is: "Build a small movie-review interface that identifies overall sentiment and explicit opinions about movie aspects, stores a local user's rating history, and recommends unseen movies using explainable features. Demonstrate distributed aggregation of MovieLens ratings using Scala and Spark."

What the demonstrator should prove

The interface should accept text, show results while the person types, distinguish opposite opinions within a review, and leave unmentioned aspects unscored. Saving a film should change the recommendation profile and remove that film from future suggestions. A separate Spark job should read rating records and generate movie and genre summaries.

What success does not require yet

You do not need a chatbot, voice input, image posters, cloud hosting, a GPU, Kafka, Kubernetes or a distributed online inference service for this checkpoint. Adding these before the baseline works makes it harder to explain where errors originate. The small implementation already exposes data cleaning, representation choices, ranking, evidence and batch processing.

A sensible project title

"CineScope: Real-Time Movie Review Analysis and Personalized Recommendation Prototype" is accurate. If your department requires big data in the title, add "with Scala/Spark Rating Analytics." Avoid presenting the current rules as a trained aspect transformer or describing a small local dataset as a large-scale performance experiment.

Mid-sem deliverable checklist

Demonstrate one mixed review, one negation example, one review with no opinions, two history updates and the resulting recommendation changes. Explain which results are rules and which are learned after optional training. Show Scala source and, when you have run it, the generated Parquet and genre aggregates. Report executed work separately from planned work.

3. Basic vocabulary without prerequisites

A dataset is a collection of examples. In this project, one example could be a written review, a numeric rating or a movie record. A feature is a useful numerical representation of an example. A label is the answer used to teach or evaluate a supervised model, such as positive or negative sentiment.

A model is a function with parameters. Training adjusts those parameters using examples. Inference applies the completed model to a new input. The default CineScope engine uses hand-written rules rather than learned parameters. This is still a valid baseline, but it should be described honestly.

The three kinds of text task

Classification chooses a category, such as positive sentiment. Extraction finds information inside the text, such as the phrase "excellent acting." Generation produces new text, such as a summary. These tasks require different labels and metrics. Good overall sentiment classification does not prove that aspect extraction or summarization is good.

Numbers that sound similar

A rating is a user's explicit preference, usually on a star scale. A sentiment score describes the language in a review. A ranking score orders candidate movies for a user. A probability is a model's numerical estimate for an event. Accuracy is the fraction of correct predictions over a labeled test set. These quantities must not be used interchangeably.

For example, a user might give a film 4/5 while writing "The ending was awful." The rating expresses the person's overall judgment; the negative clause concerns one component. A recommendation score of 0.32 means the item ranked according to the formula, not that the user has a 32% probability of enjoying it.

Four software layers

The frontend is what the user sees. The backend accepts requests and computes responses. Storage remembers state. A batch pipeline processes many stored records together. CineScope uses browser storage for personal history, Python for immediate analysis and Scala/Spark for optional dataset aggregation. Keeping these roles separate makes the flow easier to debug.

4. The architecture: online and offline paths

The online path starts when a person types into the review box. JavaScript waits for a short pause, sends the text to Python, receives JSON, and renders overall sentiment, aspect scores and evidence. Saving a rating updates browser history and requests recommendations. The Python server reads its catalog once when it starts.

The offline path starts with downloaded datasets. MovieLens ratings become movie statistics through either the small Python importer or the Scala Spark job. Labeled IMDb reviews become training examples for optional sentiment experiments. A trained TF-IDF classifier can be exported as simple JSON and loaded by the core app after a restart.

Component	Input	Output	When it runs
Browser review form	User text	HTTP request	After typing pauses
Python analyzer	Review text	Scores and evidence	Per request
Browser history	Movie, rating, review	Local saved profile	On explicit save
Recommender	Catalog and history	Ranked unseen movies	On profile refresh
MovieLens importer	Movies and ratings CSV	Small catalog JSON	Offline
Scala Spark job	MovieLens files	Parquet, CSV and bounded JSON	Offline
Optional trainer	Labeled reviews	Models and measured reports	Offline

Why not run Spark for every keystroke?

Spark is designed to schedule parallel work over datasets. Scheduling a distributed job for a short review would add overhead without a useful amount of parallel computation. The live demo needs a quick function call. Spark belongs in the historical-data preparation path, where aggregation can be distributed over many records.

Where integration really occurs

The Scala JSON export can be converted into `data/movielens.json`. The server then loads it using `CINESCOPE_CATALOG`. The optional TF-IDF trainer writes `data/sentiment.json`, which the server recognises at startup. Other research artifacts are deliberately not auto-loaded. An artifact existing on disk does not prove that the web application is using it.

5. Install and run the smallest version

Extract the ZIP. Open the `CineScope_Midsem` folder in a terminal. You should see `app.py`, `engine.py`, `web`, `data`, `optional` and `scala`. Use Python 3.10 or newer; a 3.11 or 3.12 environment is a sensible starting point for this package.

```
python app.py
```

The terminal prints the local address. Open `http://127.0.0.1:8000` in your browser and keep the terminal running. On Windows, `py app.py` may work when the `python` command is not configured.

The default app needs no package installation and no external service.

Your first five interactions

- Click "Try an example" and inspect the six aspects.
- Replace "excellent acting" with "awful acting" and observe the changed score.
- Enter "The story was not bad" to see the negation rule.
- Choose a movie and rating, then save it to history.
- Refresh the page to confirm that this browser remembers the history.

Common startup mistakes

Opening `index.html` directly is not the intended route; the page calls the Python API. Running the command from a completely different folder can also make relative environment-variable paths confusing. The default catalog path is anchored to `app.py`, but a custom relative `CINESCOPE_CATALOG` is resolved from the terminal's current directory.

If port 8000 is occupied, set the `PORT` environment variable to another port before starting. In PowerShell, use `$env:PORT = "8001"`. On macOS/Linux, use `PORT=8001 python app.py`. Stop the server with Ctrl+C.

What gets saved?

Ratings and submitted review text remain in browser localStorage. There is one local profile and one latest entry per film. The server does not create an account or a database. Clearing history or browser site data removes those preferences. Demo and MovieLens histories use separate storage keys, preventing accidental overlap between their different ID namespaces.

6. Understanding the frontend

The frontend consists of three familiar files. `index.html` declares controls and result containers. `style.css` controls layout, colors and responsive behavior. `app.js` handles browser events, API calls, history and display updates. No frontend framework or build process is required.

The browser does not train a model. It collects text and displays the result returned by Python. This makes it easier to replace the analysis engine later while leaving the interface mostly unchanged.

What "real time" means here

The application waits 300 milliseconds after the latest input event before requesting analysis. This is called debouncing. It avoids sending one request for every individual keystroke. The actual response time is the debounce delay plus computation, network and browser rendering time. There is no hard real-time deadline guarantee.

A request counter prevents an older response from overwriting newer text. When another input event happens, the counter increases immediately. Each response checks whether it still corresponds to the newest request. This matters if a later request happens to finish before an earlier one.

Displaying text safely and clearly

Review content and movie titles are escaped before being inserted into HTML. A string containing an HTML tag should be displayed as text rather than executed as page markup. Score bars are generated from numeric results, and missing scores are written as "No evidence."

Search and history

The search box filters movie titles and shows at most 200 matching options at a time. Selecting a film shows its catalog genres. Saving replaces that film's earlier history entry rather than adding duplicate ratings. Recommendations refresh using the current history; removing a film makes it eligible again.

A beginner debugging exercise

Open the browser developer tools and inspect the Network panel. Type a review and find `/api/analyze`. Read the request JSON and the response JSON. Seeing the same text travel through the API is the simplest way to connect frontend events to backend functions.

7. Backend and API contracts

The backend uses `ThreadingHTTPServer` from Python's standard library. It serves three static web files and three API routes. This is intentionally a local teaching server. It does not implement accounts, distributed state, production request management or multi-user authorization.

Route	Request	Response
GET /api/catalog	No body	Movies, dataset label and model label
POST /api/analyze	Object with text	Overall and aspect analysis
POST /api/recommend	Object with history array	Up to six candidate movies

Example analysis request

```
{"text":"The acting was excellent but the story was weak."}
```

The response contains the cleaned text, active method, overall label, overall score, opinion count and dictionaries for aspects, themes and genres. Every aspect includes a score and an evidence list. A score can be null. JavaScript must preserve that distinction: null is not the number zero.

Input boundaries

The server limits request bodies, review length and the number of history entries. Ratings must be finite numbers between 0.5 and 5 in half-star steps. Unknown movie IDs do not become recommendation history. These checks prevent many accidental malformed requests from propagating into the scoring code.

The local frontend limits review length to 10,000 characters. The backend repeats that check because a request can be sent without using the frontend. Frontend validation improves usability; backend validation defines the accepted data contract.

How errors appear

Malformed input produces a JSON error with HTTP status 400. Unknown routes produce 404. The frontend shows a short error message if an API call fails. A network failure is different from a negative sentiment prediction: one is a technical error, the other is an analysis result.

Minimal persistence choice

Because browser history is sent with each recommendation request, the server does not need SQLite. This is suitable for a small demo, but a later multi-user version should keep authenticated user data in a database and avoid trusting a client-supplied account identity.

8. Dataset roles: MovieLens is not the NLP corpus

Use MovieLens for recommendation data: movie identities, genres, ratings and rating timestamps. Use a labeled review corpus such as Stanford aclImdb for sentiment training. Use separately collected human annotations for aspect evaluation. These data sources solve different problems.

The MovieLens-small README describes four files: `movies.csv`, `ratings.csv`, `tags.csv` and `links.csv`. Its ratings use half-star increments from 0.5 to 5. Tags are short user-entered metadata, not a substitute for a labeled full-review corpus. Read the dataset's own attribution and usage conditions before redistributing it. [R1]

What the new package contains

The built-in catalog has 12 fictional films and short synthetic reviews written for the demonstration. Those reviews allow recommendation features to be inspected without downloading anything. They do not provide a valid test set for research results. The titles and review evidence are clearly identified as authored demo data.

The Python importer reads MovieLens movies and rating events, calculates average ratings and counts, and creates a compatible JSON catalog. It can also accept a movie-linked review CSV that you supply. Without linked review text, the recommendation engine simply has no candidate-side review themes or aspect quality signals.

Three identities to keep separate

A MovieLens movie ID identifies a movie. A MovieLens user ID identifies an anonymized rating account within that dataset. Your local browser profile identifies neither. The demo does not claim that you are a particular MovieLens user, and your saved ratings are not joined to their historical user IDs.

A valid data statement

"MovieLens provides metadata and rating statistics. User-entered reviews support live analysis. Optional IMDb data train the overall sentiment classifier. Aspect labels are collected separately." This sentence is much more accurate than saying that MovieLens trains every NLP feature.

9. IDs, joins and trustworthy data integration

An ID is a stable reference used to connect records. Titles are poor join keys: spellings, translations, release years and punctuation can differ. Two films can also share a title. Use documented mapping files whenever possible.

MovieLens `links.csv` connects its movie IDs to external IMDb and TMDB identifiers. The bundled importer does not automatically download external reviews or guess movie mappings. A user-supplied review CSV must already contain valid MovieLens `movieId` values.

Example of a valid local join

```
movies.csv:          movieId,title,genres
linked_reviews.csv:  movieId,review
```

The importer groups review text by `movieId`, then attaches those groups to the movie record with the same key. A review with an ID absent from the catalog should be inspected as an unmatched record. In a final pipeline, produce a join report with input rows, matched rows, unmatched rows and duplicate keys.

The dangerous shortcut

Do not treat an IMDb review filename such as `123_8.txt` as MovieLens movie 123. Its first number is a review-file index in a particular corpus partition, not a MovieLens identifier. The supplied IMDb preparation script uses matching URL files to recover title groups when available, but does not attempt to map them into MovieLens automatically.

Provenance fields

For real review ingestion, retain a source name, source record ID, movie ID, retrieval date and text hash. If reuse conditions permit retaining the text, preserve the original separately from cleaned text. You then know which transformation produced the text sent to the model.

The small app caps imported reviews at 20 unique texts per movie to keep the browser catalog and recommendation work bounded. That is a serving simplification, not a rule for research sampling. Perform corpus experiments and evaluation on a separately documented dataset, not silently on whichever 20 rows happened to arrive first.

10. Text preprocessing, step by step

Raw reviews can contain HTML, entities, punctuation, contractions, emojis, slang, repeated text and spelling mistakes. Preprocessing makes the representation consistent while trying to preserve meaning. The correct choices depend on the downstream task.

The core `clean()` function unescapes HTML entities, removes simple markup, lowercases English text, expands common negative contractions, maps a few explicit slang and emoji examples, and normalizes whitespace. This is a small deterministic baseline, not a complete language-normalization system.

Input	Intended cleaned effect	Limitation
`<p>Great acting</p>`	great acting	Regex HTML stripping is basic
`wasn't good`	was not good	Not a full contraction grammar
`gr8 music`	great music	Tiny hand-written slang map
Happy mapped emoji	Positive lexicon token	Many emojis remain unknown
`amazzing acting`	Misspelling retained	No spelling corrector is included

Why keep negation?

If "not" is removed, "not good" becomes "good," reversing the opinion. General stop-word removal is therefore unsuitable for this rule engine and the chosen sentiment vectorizer. A word that is common can still be crucial to the label.

Tokenization

The tokenizer extracts English alphabetic tokens, retaining some apostrophe forms. It is simple and predictable but not multilingual. It does not model subword pieces, hashtags or every stylized spelling. The transformer experiment uses its own model tokenizer after the prepared text, so its input representation differs from the count-based baseline.

Spelling and slang policy

For the mid-sem demonstration, document failures instead of adding dozens of untested corrections. A correction such as "cast" to "cost" would damage aspect extraction. If a later spelling component is introduced, measure both recovered sentiment words and newly introduced mistakes on a manually reviewed sample.

Useful manual test set

Include a contraction, an emoji, repeated punctuation, an HTML break, a misspelling, a mixed-language sentence and an empty review. Record what the current implementation actually does for each. These examples expose the preprocessing boundary better than a single perfectly written sentence.

11. Sentences, clauses and duplicate handling

A sentence can contain several different opinions. "The acting was excellent but the story was weak" is one sentence with two contrasting clauses. Splitting it at "but" lets the small engine handle each local opinion independently.

The implementation splits normalized text at sentence punctuation and selected contrast words such as "but," "however" and "although." This is a heuristic clause splitter. It does not reconstruct the full grammatical structure, handle every abbreviation or understand all subordinate clauses.

Why splitting helps

Suppose an overall positive review contains one negative ending comment. If every aspect receives the whole-review label, the ending will incorrectly appear positive. Local clauses reduce this problem by narrowing the text associated with an aspect. Nearest-opinion assignment then narrows it further.

Exact duplicates versus near duplicates

Exact duplicate detection compares the same normalized text or its hash. Near duplicates differ slightly: extra punctuation, a copied paragraph with one word changed, or a reposted review with a new heading. Exact matching is cheap and auditable. Near matching requires a similarity rule and a threshold whose false positives must be checked.

The IMDb preparation script hashes cleaned text, removes duplicate copies and drops text with conflicting sentiment labels. When an exact duplicate appears in both official partitions, it retains the test copy and removes the training copy. This creates a deduplicated variant, so report the resulting counts rather than claiming an untouched official benchmark.

What the small app does

History has one record per movie, with a new save replacing the old entry. The MovieLens review importer deduplicates exact stripped review strings per film. There is no universal spam detector and no claim to identify deceptive reviews. Duplicate content is evidence of repetition, not proof that a reviewer is dishonest.

Research exercise

Before and after deduplication, count rows, unique hashes and label balance. Audit a few collisions manually. A model's score can fall after leakage is removed; that is a more credible measurement, not a reason to restore duplicate leakage.

12. Morphology: stemming versus lemmatization

Morphology concerns word structure and word forms. Reviews contain related words such as "act," "acting," "acted" and "actors." Reducing variation can make a feature vocabulary smaller, but aggressive normalization can also remove meaningful distinctions.

Stemming applies rules that trim word endings. A stem is not guaranteed to be a dictionary word. Lemmatization attempts to return a dictionary base form, often using part of speech. "Was" can map to "be," while a stemmer may leave it unchanged. Exact outputs depend on the chosen library and version.

Why context matters

"I saw the film" uses "saw" as a verb. "The saw was loud" uses it as a noun. A lemmatizer given the correct part of speech can treat these differently. A context-free string rule cannot reliably do so.

A controlled project experiment

Use the same train, validation and test IDs for three pipelines: no morphological reduction, stemming and lemmatization. Keep the vectorizer, n-gram range, feature cap and classifier tuning budget unchanged. Fit every learned preprocessing artifact on training data only. Report vocabulary size, macro-F1 and runtime.

Variant	Question to answer
Original tokens	Is preserving surface form already adequate?
Stemmed tokens	Does a smaller vocabulary offset lost distinctions?
Lemmatized tokens	Does linguistic normalization improve generalization?

Expected trade-offs, not promised results

Stemming can merge useful variants, but unusual fragments may make explanations less readable. Lemmatization can preserve more meaningful forms, but needs linguistic resources and may fail on slang or typos. A pretrained transformer usually expects its original tokenization pipeline; inserting stemming can move its input away from the distribution it learned.

This comparison is documented as an extension. The core code does not secretly lemmatize text, and the optional training scripts do not claim to have completed this ablation. For your final report, include actual before/after token examples and measured results rather than saying lemmatization is always superior.

13. Overall rule-based sentiment: the exact calculation

The simplest sentiment baseline maintains a positive-word set and a negative-word set. Each detected positive word contributes +1; each negative word contributes -1. A short negation rule can flip that contribution. The engine combines all opinion hits across clauses.

```
score = 50 + 50 * (sum of signed opinion values / number of hits)
```

If there are no opinion hits, the score is null. The label is positive above 60, negative below 40, and mixed/balanced between those boundaries. These thresholds are implementation choices, not experimentally optimized claims.

Worked example

"The acting was excellent but the story was weak" contains one +1 hit and one -1 hit. Their sum is zero and the number of hits is two. The overall score is 50. The positive acting and negative story remain separate in the aspect results.

"Great visuals and beautiful music" contains two +1 hits. The score becomes 100. This says only that the detected opinion words agree positively. It does not establish perfect confidence, objective movie quality or 100% classification accuracy.

Strengths

The method is tiny, deterministic and easy to explain. It runs without training data and lets the user inspect the lexical assumptions. It is useful as a baseline and as a way to debug the interface before model training.

Weaknesses

The word "slow" is treated negatively, although a reviewer may praise a slow and thoughtful film. "Great, another pointless sequel" contains sarcasm that the rules cannot fully understand. Words outside the lexicon are ignored. A review can therefore be scored from a small, unrepresentative subset of its meaning.

Honest reporting

Call this a heuristic sentiment index. Measure its actual performance on human-labeled reviews if you want to report accuracy. Compare it against a majority-class baseline and a learned classifier. A few demonstration examples verify behavior but do not estimate generalization to unseen real reviews.

14. Negation, intensity and mixed feelings

Negation reverses or qualifies an opinion. The small engine looks up to three tokens before an opinion word for "not," "no," "never," "hardly" or "barely." An odd number of those tokens flips the sign. This is a transparent local approximation.

Examples to trace

"The acting was not good" turns the positive hit for "good" into a negative contribution. "The story was not bad" turns "bad" into a positive contribution. The latter is not necessarily strong praise in real language, but the binary sign rule treats it as positive.

"Not only was the acting good, the music was excellent" illustrates a failure: "not only" is a construction whose meaning is not simple negative polarity. A larger window can catch distant negation but also produce more incorrect flips. A shorter window misses longer grammatical relationships.

Intensifiers are not learned here

"Good" and "extremely good" receive the same sign in this version. The implementation does not assign a special multiplier to "very," "slightly" or "absolutely." You could add an intensity dictionary, but then you should document how multiple modifiers combine and evaluate the change on held-out examples.

Balanced score versus neutral meaning

A score of 50 may result from one positive and one negative opinion. That is mixed evidence. A sentence such as "The movie lasts two hours" has no opinion evidence and gets null. A learned neutral class requires labeled neutral examples; a balance of positive and negative lexicon counts is not equivalent.

An extension worth evaluating

Use dependency relationships to determine whether a negator modifies the adjective or verb attached to an aspect. Compare this against the three-token rule on sentences specifically labeled for negation. Report both the corrected cases and the newly broken cases. Context-sensitive models can help, but still require evaluation rather than an assumption of perfect understanding.

15. Six aspects: from words to local evidence

An aspect is a part of the movie that a reviewer judges. CineScope includes Story, Acting, Visuals, Music, Pacing and Ending. Each aspect has a short alias list. For example, Story includes "plot" and "script," while Visuals includes "cinematography" and "effects."

The engine locates aspect keywords and opinion words inside each clause. It assigns an opinion to the nearest aspect anchor within four tokens. If the closest distance is tied between different aspects, it skips that assignment. Skipping uncertainty is preferable to confidently giving the same opinion to unrelated aspects.

Worked trace

In "The acting was excellent," the anchor "acting" occurs near the opinion "excellent." The opinion contributes +1 to Acting, which receives 100. In "The story was weak," the negative hit contributes -1 to Story, which receives 0. Music, Visuals, Pacing and Ending remain null because there is no local scored evidence.

The evidence field contains the normalized supporting clause. It is not an original-character offset annotation. Because cleaning lowercases and expands some tokens, evidence text may differ from the user's exact original formatting.

Important limitations

"It looked magnificent" may imply visual quality without an explicit recognized aspect term. "The actors were good and the story was terrible" can still challenge proximity rules when anchors and opinion words interleave. A distance threshold cannot replace grammatical understanding.

Pacing words such as "slow" are both aspect clues and lexical opinions. This convenient shortcut may misclassify deliberate slow pacing. A final model should separate detecting an aspect from determining the sentiment expressed about it.

Evaluation target

Create human labels for whether each aspect is mentioned and whether the expressed sentiment is positive, negative, neutral or mixed. Evaluate each aspect separately. An overall sentiment label should never be copied onto all six aspects. That would create incorrect supervision for reviews with mixed opinions.

16. Theme and genre scores: what they really mean

A genre is a catalog category such as Comedy or Sci-Fi. A theme is a recurring idea such as friendship, identity or survival. An aspect is a craft component such as acting or music. Keeping these concepts separate prevents misleading charts.

The UI displays a film's catalog genres independently from genre mentions detected in the review. A film may be classified as Sci-Fi even if its review never uses the phrase "science fiction." Failure to mention a genre is not evidence that the metadata is wrong.

What a detected tag contains

For each explicitly mentioned theme or genre alias, the engine collects matching clauses. It calculates the sentiment index from all opinion hits in those clauses and reports clause coverage. Coverage is the number of matching clauses divided by all nonempty clauses, multiplied by 100.

If one out of four clauses mentions friendship, Friendship has 25% coverage. This is not a 25% probability that the film concerns friendship. If the matching clause includes a positive opinion, it can have a high sentiment index despite low coverage. Multiple tags can share a clause, so their coverage percentages need not sum to 100.

A subtle limitation

"The friendship theme was interesting, but the acting was bad" is relatively easy because the contrast splits the text. "The friendship theme appears while the acting is awful" can attach negative clause sentiment to Friendship even though the opinion targets acting. These are mention-conditioned clause scores, not a trained theme-specific sentiment model.

How to improve later

Annotate theme mention spans and opinion targets. Add dependencies or a supervised target-conditioned model. Evaluate theme detection separately from theme polarity. Use movie metadata to describe genre membership, and use actual review evidence to describe opinions. Never label an entire genre objectively bad based on one user's sentence about one film.

17. Syntax and semantics: beyond nearby words

Syntax describes grammatical relationships; semantics concerns meaning. A proximity rule uses token distance, while a syntactic parser can tell you which adjective modifies which noun. This distinction matters when a review discusses several movie aspects in the same sentence.

Part-of-speech tagging assigns labels such as noun, verb and adjective. Dependency parsing links words through grammatical relationships. In "excellent acting," an adjective modifies the acting noun. In "the story is weak," the structure links the description to the story even though another word appears between them.

A practical extension

Run a linguistic model over the original clause. Locate aspect nouns or their aliases. Collect adjectival modifiers, predicate adjectives, relevant opinion verbs and attached negators. Return both the aspect and an evidence span. Keep rules for coordinated phrases such as "great acting and music," where one adjective can apply to more than one target.

Why this is still imperfect

Parsers can fail on fragments, slang, misspellings and unconventional punctuation. Correct syntax also does not guarantee correct sentiment: "unpredictable" can be praise for a thriller but criticism of a user interface. Domain and context determine meaning.

The right comparison

Build a manually labeled set with simple adjacency, long-distance opinions, coordination, negation and implicit aspects. Compare the current nearest-aspect method with the dependency method. Keep the review IDs and annotation definitions fixed. Report aspect detection F1, sentiment F1 on matched targets, evidence-span quality and runtime.

Current implementation boundary

The packaged core uses nearest-aspect rules. A POS tagger or dependency parser is not silently running behind the interface. This chapter specifies the next integration, including what its output must contain and how to determine whether it actually improves the project. Optional linguistic demonstrations alone do not count as an integrated extraction component.

18. One-hot vectors, BoW and TF-IDF

Computers need numerical representations of text. Start with a vocabulary: an ordered list of tokens found in training data. If the vocabulary is [good, bad, acting], the token "acting" has one-hot representation [0, 0, 1]. A one-hot vector represents one category, not a complete review's word frequencies.

Bag of Words, or BoW, counts how often each vocabulary term appears in a document. "good good acting" becomes [2, 0, 1]. This representation discards word order unless multiword features are included. It is sparse because most documents use only a small fraction of the whole vocabulary.

TF-IDF intuition

Term frequency measures how often a term occurs in the document. Inverse document frequency discounts terms that occur in many documents. A distinctive phrase may help more than a nearly universal token. The optional trainer uses scikit-learn's smoothed IDF and L2 normalization. [R3]

```
idf(t) = log((1 + training_document_count) / (1 + document_frequency(t))) + 1
raw_weight(t,d) = count(t,d) * idf(t)
normalized_vector = raw_vector / its_Euclidean_length
```

Small worked example

Suppose three training documents contain "acting," but only one contains "awful." The latter has higher IDF under this formula. That does not make "awful" objectively important in every task; it simply gives rare terms more relative weight in this representation. The classifier still learns how each feature relates to the target label.

Why fit only on training data?

Vocabulary membership and document frequencies are learned statistics. Fitting the vectorizer on test documents gives the pipeline information it should not have during training. Put the vectorizer and classifier in one fitted pipeline and transform validation/test data with the training-fitted vectorizer.

What the experiment controls

The supplied BoW and TF-IDF models share preprocessing, tokenization, unigrams/bigrams, a 30,000-feature cap and the same data rows. Their weighting differs. Validation selects regularization strength independently for each method. This is a meaningful comparison rather than two unrelated pipelines with many simultaneous changes.

19. Classification n-grams are not language models

An n-gram is a consecutive sequence of n tokens. A unigram is one token, a bigram contains two and a trigram contains three. "not good" is a bigram that can preserve useful sentiment context that the isolated word "good" loses.

A TF-IDF classifier can treat "not good" as a feature. The learned classifier weight tells you whether that feature tends to support a label. It does not provide a normalized distribution over the next word in a sentence.

Two different questions

A sentiment classifier asks: given a complete text, how should it be labeled? A language model asks: given preceding context, how probable is the next token? Both can use bigram counts, but their objectives, normalization and evaluation differ.

Property	N-gram classifier features	N-gram language model
Output	Sentiment prediction	Next-token probability
Supervision	Sentiment labels	Token sequences
Typical metric	Macro-F1, accuracy	Held-out perplexity
Counts used for	Input features	Conditional distributions
Smoothing	Not required to define a label	Needed for unseen sequences

Example

A classifier may learn a strong negative coefficient for "waste time." A bigram language model might estimate the probability of "time" following "waste." Those values cannot be compared directly: one is a parameter in a decision function; the other is a conditional probability with a defined denominator.

Project mapping

`train_baselines.py` uses unigram and bigram features for sentiment. `ngram_lab.py` separately implements actual conditional token probabilities, next-word suggestions and held-out perplexity. `Ngram_Lab.ipynb` provides a notebook entry point. Keeping the experiment separate makes its purpose easy to explain and resolves the original gap in the syllabus coverage.

20. Logistic regression from first principles

Logistic regression is a linear classification model, despite the word "regression" in its name. It combines features using learned weights, adds an intercept and passes the result through a sigmoid function.

```
z = intercept + sum(weight_j * feature_j)
p(positive | review) = 1 / (1 + exp(-z))
```

Positive weights increase the positive-class tendency; negative weights decrease it. If z is zero, the probability is 0.5. If z is large and positive, the probability approaches one. Training adjusts weights to reduce classification loss while regularization discourages excessively large parameters.

Worked numerical example

Suppose a two-feature review vector is [0.8, 0.2], the weights are [2, -3], and the intercept is -0.1. The linear score is -0.1 + 20.8 - 30.2 = 0.9. The sigmoid is about 0.711. With a 0.5 threshold, the prediction is positive.

This calculation is not an accuracy measurement. Accuracy needs many labeled held-out examples. Even a high model probability can be wrong, particularly when the text differs from the training distribution.

Why it fits mid-sem work

A TF-IDF logistic-regression model trains relatively cheaply, has understandable features and often provides a useful text-classification baseline. It lets you study representations and evaluation before adding GPU-dependent models. The delivered app can load its JSON coefficients without importing scikit-learn at serving time.

Regularization

The optional trainer tests C values 0.5 and 2.0 on validation data. In this model, smaller C means stronger regularization. The selection is made separately for BoW and TF-IDF. Test scores must not be used to pick whichever parameter looks best after the fact.

Inspecting errors and weights

Look at top positive and negative coefficients, but interpret them with care. Correlated tokens can share information, and a token's weight depends on the rest of the fitted representation. Error analysis on actual reviews is more informative than a list of apparently impressive keywords.

21. Preparing IMDb without leaking labels

The optional preparation script expects an extracted `aclImdb` directory containing labeled `train/pos`, `train/neg`, `test/pos` and `test/neg` folders. It assigns label 1 to positive reviews and 0 to negative reviews. Unlabeled examples are not used by this supervised baseline.

The Stanford dataset is a movie-review sentiment benchmark with predefined training and test partitions. Its source page provides download and attribution information. The small project uses it for overall polarity, not as a ready-made six-aspect dataset. [R2]

What the script writes

`data/reviews.csv` contains `id`, `text`, `label`, `split` and `group`. The ID is a SHA-256 hash of cleaned text. The group is an IMDb title ID recovered from URL metadata when available; otherwise it is the text hash. `data/split_manifest.json` records counts and the splitting policy.

Duplicate and split policy

The script processes official test examples first so a duplicate overlapping training is retained only in the test set. Reviews with conflicting labels for identical cleaned text are removed. Within the official training partition, a deterministic hash of the movie group assigns approximately one fifth of groups to validation and the rest to training.

Because grouping operates on movies, the validation fraction is approximate rather than exactly 20% of reviews. Check class balance and split sizes. The script does not enforce disjoint movie groups between the official test partition and the training partition; inspect that overlap and report the policy accurately.

What to record

Save dataset provenance, original file counts, removed duplicates, conflicts, split counts, label balance and the script version. Deduplicated results should be described as a transformed benchmark rather than the untouched official test set. If you alter the splitting policy, rerun every compared model on the same new manifest.

Avoid silent subsampling

If your laptop needs a smaller subset, choose it once from each split, save the selected IDs and reuse them across experiments. Do not train the transformer on a convenient subset and compare its score to a baseline measured on unrelated test reviews without clearly stating the difference.

22. A controlled BoW versus TF-IDF experiment

An experiment changes a specific factor while holding the rest of the setup as constant as practical. Here the central factor is how word counts are weighted. Both pipelines use the same cleaned texts, tokenizer, unigram/bigram range, vocabulary budget and training rows.

Procedure

Install the optional requirements, prepare IMDb once, and run `python optional/train_baselines.py`. For each representation, the script fits two regularization settings using training data and selects the one with higher validation macro-F1. It then reports final held-out test metrics for the selected model.

The script writes a report JSON and one prediction file per representation. Each prediction file preserves review IDs alongside gold and predicted labels. Keeping individual predictions allows paired error analysis and later significance checks; a single accuracy number cannot support those analyses.

Reporting table to fill after execution

Model	Validation macro-F1	Test macro-F1	Test accuracy	Time
Lexicon	Measure separately	Measure separately	Measure separately	Measure
BoW + LR	Generated by run	Generated by run	Generated by run	Measure
TF-IDF + LR	Generated by run	Generated by run	Generated by run	Measure

The empty cells are intentional. They are not estimated results. The package includes no completed real-corpus training run for these new scripts.

What could change the conclusion?

A smaller vocabulary, different n-gram range, different regularization search or domain shift can alter which representation wins. If TF-IDF performs better, explain that it was better under this setup, not universally better for all sentiment tasks.

The serving artifact is TF-IDF by design, regardless of whether BoW's test score happens to be higher. This prevents using the test set as an undeclared model-selection set. If you want automatic model-family selection, compare validation scores, freeze the choice, then make one final test evaluation.

23. Deploying a trained baseline without a heavy server

The trainer exports a JSON object containing the vocabulary, IDF values, classifier weights and intercept. The core Python server loads it from `data/sentiment.json` at startup. It reproduces token counts, unigram/bigram features, TF-IDF weighting, L2 normalization and the sigmoid calculation.

Why export JSON?

For this specific linear model, the inference equations are short. A JSON artifact lets the core app remain free of training dependencies. It also makes the serving parameters inspectable. This is not a universal substitute for model serialization; a transformer cannot be reduced to the same small list of coefficients.

Consistency requirement

Training and serving must use the same tokenizer, normalization, feature spelling, vocabulary indices and IDF values. If training treats "not good" as one bigram feature while serving builds a different token sequence, predictions can silently diverge. Verify probabilities against the original fitted pipeline on representative inputs after changes.

How the interface changes

When the file exists, the overall score becomes 100 times the model's positive-class probability. The method label explicitly says TF-IDF + Logistic Regression, overall only. Aspect, theme and genre calculations still use the rule engine. A learned overall classifier does not automatically turn rule-based aspect extraction into a trained aspect model.

Practical sequence

Stop the app, run training, inspect the measurement files and restart. Confirm the model label in the interface. To restore the rule-only baseline, rename or remove the JSON and restart. The model is loaded once, so changing the file while the server is already running does not update the active parameters.

Evaluation boundary

The included numerical smoke check verifies inference consistency for a small controlled fixture. It does not establish real IMDb accuracy. Accuracy, calibration and robustness require the real held-out data and saved experiment report.

24. A genuine bigram and trigram language model

The optional n-gram model estimates the probability of a token from its preceding context. A bigram uses one preceding token; a trigram uses two. It counts observed context-token combinations in training data and turns them into conditional probabilities.

```
P(word | context) = (count(context, word) + alpha)
                    / (count(context) + alpha * vocabulary_size)
```

The supplied implementation uses alpha = 1, called add-one or Laplace smoothing. Smoothing gives nonzero probability to unseen combinations. Without it, one unseen test bigram would make a sentence probability zero and perplexity infinite.

Vocabulary and boundaries

Training words occurring fewer than twice map to <unk>. The vocabulary includes an end token and an unknown token. Start markers provide initial context; they are not predicted output tokens. Unseen test words map to the unknown token using the training vocabulary.

Worked example

Suppose the context "movie" was followed by "is" three times and by "was" once. With vocabulary size five and alpha one, P(is | movie) = (3+1)/(4+5) = 4/9. An unseen continuation receives 1/9. Probabilities across all five possible output words sum to one.

Why this lab is separate

It teaches conditional probability, vocabulary decisions, smoothing and sequence evaluation. The live sentiment UI does not need to predict the next word, so the language-model code stays outside the core application. This keeps a required syllabus experiment from making the demo unnecessarily complicated.

Limitations to explain

Count models struggle with long-distance relationships and sparse contexts. Add-one smoothing is easy to understand but often suboptimal. A final extension can compare other smoothing approaches, but must keep tokenization and evaluation vocabulary consistent to make perplexity comparisons interpretable.

25. Perplexity and next-word prediction

Perplexity summarizes how surprised a language model is by held-out tokens. Lower perplexity is better only when the models are evaluated on the same tokenization, vocabulary policy and test sequence. It is not a sentiment-classification metric.

```
average_negative_log_probability = -sum(log P(test_token | context)) / N
perplexity = exp(average_negative_log_probability)
```

N includes the predicted end-of-sequence tokens in the supplied implementation. The calculation uses log probabilities to avoid underflow from multiplying many tiny numbers.

Numerical example

If the model assigns probability 0.5 to every observed test token, the average negative log probability is log(2), so perplexity is 2. If it assigns probability 0.25 to every token, perplexity is 4. Real models assign different probabilities at different positions, so perplexity is a geometric aggregation, not a simple arithmetic average.

What the script produces

The script fits both a bigram and a trigram model on training reviews, evaluates held-out perplexity, and lists likely next words after "the movie." It also records vocabulary size and smoothing strength. The notebook first demonstrates a tiny toy corpus, then provides a cell to run the real-corpus experiment.

A common surprising result

The trigram model can perform worse than the bigram model on a small dataset because its longer contexts are observed less often. More context is useful only when the estimation method and data can support it. This is a concrete example of the sparsity problem.

Suggested extension

Choose smoothing strength using validation perplexity rather than test perplexity. Keep the test set untouched until the choice is fixed. Save per-review log loss for error analysis: unusually high values can reveal domain mismatch, tokenization problems or many unknown words.

26. Collocations: frequency and PMI

A collocation is a phrase whose words occur together in a meaningful way, such as "powerful performance" or "predictable ending." Frequent word pairs are easy to find, but frequent function-word pairs may be less interesting than domain-specific phrases.

Pointwise Mutual Information, or PMI, compares the observed probability of a pair with what would be expected if its words occurred independently. The corpus lab uses adjacent bigram counts and matching left/right positional marginals.

```
PMI(a,b) = log2( count(a,b) * total_bigrams
                / (left_count(a) * right_count(b)) )
```

A high positive value indicates stronger-than-independent co-occurrence under these counts. PMI is not sentiment and does not automatically say whether the phrase is useful for recommendation.

Worked intuition

If "powerful" appears very often before "performance," the pair can have high PMI. But a rare typo repeated once can also look strongly associated. This is why the script requires a minimum bigram frequency of ten before ranking phrases.

What the supplied experiment does

`corpus_lab.py` uses the real training reviews from the shared CSV, extracts adjacent bigrams, computes PMI and writes the top phrases. It does not use the test corpus to choose phrases. The current version includes simple cross-clause adjacency after tokenization, so manually inspect boundary artifacts.

How to improve it

Restrict candidate pairs to adjective-noun or noun-noun sequences after POS tagging. Compare frequency-only ranking with PMI plus a minimum count. Keep examples of accepted and rejected phrases. If phrase features enter a classifier, learn the phrase inventory on training data only and evaluate the effect as a separate ablation.

What to report

Include counts beside PMI, show ten understandable phrases, explain your filtering rule, and discuss nonsense outputs. A topic or phrase table is useful only when you interpret what the model actually discovered rather than assigning attractive labels without checking examples.

27. Word2Vec: CBOW and Skip-gram

Word embeddings represent words as dense vectors learned from surrounding text. Words appearing in related contexts often develop similar vectors. This captures a different kind of information from one-hot vectors, where unrelated and related words are equally separated.

CBOW predicts a target word from nearby context words. Skip-gram predicts nearby context words from the target. They are different training objectives within the Word2Vec family; the Gensim implementation exposes the choice through its `sg` parameter. [R4]

Project experiment

`word_vectors.py` trains both methods on the shared training reviews. It uses 100 dimensions, window size five, minimum frequency five, five epochs, one worker and seed 42. It saves the vectors and lists neighbors for "story," "acting," "excellent," "boring" and "music" when those words exist.

These choices are a small reproducible starting point, not an optimized configuration. Setting a random seed helps, but library versions and hashing can still affect results. Set `PYTHONHASHSEED=42` before execution and record the software environment if exact repeatability matters.

Cosine similarity

Cosine similarity compares vector direction: dot(a,b) divided by the product of their lengths. A value near one means similar direction. It does not mean the words are synonyms or have the same sentiment. "Good" and "bad" can occur in similar grammatical contexts and therefore be close in an embedding space.

A useful report

Compare the neighbors produced by CBOW and Skip-gram for the same query words. Mark unknown words and report vocabulary size. Inspect rare words separately from frequent words. Add a small human-rated similarity set if you want a quantitative comparison.

Practical limit

Averaging word vectors into a document vector loses word order and much of negation. "Good" and "not good" can have very similar averages. Word2Vec is an important representation experiment, but it does not by itself solve target-specific sentiment or conversational understanding.

28. GloVe comparison and embedding experiments

GloVe is another word-vector approach, based on global word co-occurrence statistics. Stanford distributes pretrained vectors through its project page. The optional script accepts a local GloVe text file; it does not bundle or automatically download a large pretrained artifact. [R5]

Keep the comparison interpretable

Use the same query words and, preferably, the same embedding dimension when inspecting neighbors. The supplied command example uses a 100-dimensional GloVe file against 100-dimensional locally trained Word2Vec vectors. Even then, the methods have different training corpora and corpus sizes, so the comparison is not a controlled architecture-only experiment.

A pretrained model may understand general vocabulary well but miss movie-review slang. A small domain-trained model may capture "cinematography" and "soundtrack" relationships while lacking reliable vectors for rare terms. Neither outcome establishes universal superiority.

Quantitative options

Create a fixed list of word pairs with human similarity judgments and calculate rank correlation between model cosine similarity and those judgments. Alternatively, average each review's known word vectors, train the same sentiment classifier on each representation, and compare on the shared test IDs. Report out-of-vocabulary coverage alongside predictive metrics.

Joining embeddings to recommendation

For a later extension, build a review embedding per movie using only information available before the recommendation cutoff. Build a user vector from liked movies, subtracting or downweighting disliked ones. Rank candidate vectors by similarity, then combine with genres and rating priors using validation-selected weights.

Current project boundary

The live app uses explicit review themes and aspect indices, not Word2Vec or GloVe vectors. The word-vector script is an offline experiment that produces neighbors and saved vectors. Integrating them into ranking requires an additional feature-export and evaluation step; it should not be claimed merely because vectors have been trained.

29. DistilBERT training and fair comparison

A transformer builds contextual representations: the representation of a word depends on the surrounding text. DistilBERT is a smaller distilled BERT-family model that can be fine-tuned for classification. The optional experiment loads a pretrained model, adds a two-class prediction head and updates it on your training reviews.

The implementation uses Hugging Face model loading/saving APIs and a direct PyTorch training loop. It saves the checkpoint with the best validation macro-F1, reloads that checkpoint, and evaluates the test set. [R6]

Commands and resources

Install `torch` and `transformers` in the optional environment, then run `python optional/train_distilbert.py --epochs 2`. A GPU is useful, but the script can run on CPU at much lower speed. Initial weights require a download. This deliverable does not include a completed transformer training run.

What is held constant

The model reads the same `data/reviews.csv` and preserves the same test review IDs as the BoW/TF-IDF experiment. Its tokenization differs because it uses the pretrained model tokenizer. The script truncates to 256 tokens, whereas the TF-IDF baseline uses full prepared text. Report this difference; do not hide the lost ending of long reviews.

Training-loop intuition

For each batch, tokenize and pad examples, compute the classification loss, backpropagate gradients, clip gradient norm and apply an AdamW update. At the end of each epoch, turn off training behavior and measure validation predictions. Test results are obtained after choosing the checkpoint, not used to choose it.

A better final comparison

Measure accuracy, macro-F1, class-level recall, runtime, memory, model size and inference latency. Compare errors on negation, sarcasm, long reviews and mixed aspects. A transformer can improve classification while costing more to train and serve. Your conclusion should reflect both quality and practical cost.

30. Human aspect annotations: the missing foundation

A review-level positive label does not tell you the sentiment of every aspect. "Brilliant acting, weak story" is the simplest counterexample. A trained aspect system therefore needs aspect-specific labels and a clear annotation policy.

The template contains `review_id,text,aspect,label,split`. The allowed aspects are the six names used by the app. The labels are positive, negative, neutral, mixed and not_mentioned. The train/validation/test split should be assigned at the review level before producing multiple aspect rows.

Label definitions

- Positive: clear praise of the specified aspect.
- Negative: clear criticism of the specified aspect.
- Neutral: the aspect is discussed without evaluative polarity.
- Mixed: both praise and criticism target the same aspect.
- Not mentioned: the text contains no supported reference to that aspect.

A neutral aspect is different from an absent aspect. "The soundtrack uses piano" mentions Music neutrally. "The actors were excellent" does not mention Music.

Annotation process

Start with a pilot sample of real reviews. Have two people annotate independently using the same written guide. Compare disagreements, clarify the guide and adjudicate labels. Preserve the original independent labels if you plan to calculate agreement. Increase the dataset gradually and track class counts for every aspect.

Sample-size honesty

There is no universal number that guarantees a good model. A few dozen examples can reveal workflow problems but provide unstable estimates, especially for rare labels such as mixed. Choose a manageable first annotation round, inspect support and confidence intervals, then add targeted examples where coverage is weak.

Evidence spans

For a stronger final dataset, store exact original-text character offsets for the supporting phrase. Keep original text immutable so offsets remain valid. The basic CSV intentionally starts smaller; it supports aspect-label training but does not itself evaluate evidence-span extraction.

31. Training and evaluating six aspect classifiers

The optional aspect script trains one TF-IDF logistic-regression classifier per aspect. Every classifier reads complete review text and predicts the sentiment label for its specific target. This is a straightforward target-specific baseline, not a transformer sequence-tagging model.

Before training, the script checks that a review ID or identical cleaned text does not appear across different splits. It rejects invalid aspect names and labels. For an aspect with fewer than two training classes or no test rows, it writes an "insufficient annotations" status instead of inventing a result.

Outputs

The script saves a classification report for each aspect, including precision, recall, F1 and support across the five allowed labels. It also saves model files for later research work. These joblib files are not automatically loaded by the small frontend; only load artifacts you created and trust.

Why per-aspect metrics matter

Acting might be common and easy to detect while Ending is rare. Averaging everything together can hide a poor Ending classifier. Report both aspect-level metrics and macro averages, and show the number of test examples behind every result.

A possible stronger architecture

Encode the review together with the aspect name, then classify sentiment for that target. Alternatively, first extract aspect spans, then classify each extracted span in context. The second approach introduces an extra failure stage: missing an aspect prevents correct polarity classification even if the polarity model is strong.

Avoid an evaluation shortcut

Do not evaluate only on sentences where a keyword rule already found the aspect unless that restricted setting is explicitly the task. Such filtering can remove difficult implicit mentions and inflate apparent performance. Evaluate mention detection over all relevant inputs, and separately report polarity on gold targets or correctly detected targets.

Serving integration decision

Keep the rule system as a fallback until the trained model is evaluated. Add an explicit method label and preserve null/absent behavior. A new model should replace a component only when its output contract is clear and its limitations are documented.

32. Topic modelling with LDA

Topic modelling discovers recurring word patterns without requiring sentiment labels. LDA represents each document as a mixture of topics and each topic as a distribution over words. The topic numbers have no intrinsic human meaning; interpretation comes from inspecting words and representative documents.

The corpus lab fits six topics using a count matrix from the real training reviews. It removes a general English stop-word list for this topic experiment, uses a bounded vocabulary and records the most weighted words for each topic. These preprocessing choices differ from sentiment because the task is different.

Topic versus theme

A topic might contain actor names, plot words or release-related discussion. A hand-defined theme such as Family has an explicit taxonomy meaning. An LDA topic should not automatically be renamed Family just because one of its top words is "father." Inspect multiple high-weight reviews and check whether the label fits consistently.

Evaluation

Topic coherence can summarize whether top words tend to appear together meaningfully. Human interpretation remains useful: ask reviewers to name the topic, identify an intruder word or rate whether representative documents match the proposed label. Record topic stability across random seeds and different topic counts.

A worked analysis workflow

Fit the model using training reviews. Export top words and several high-probability documents for each topic. Propose short labels and explain supporting evidence. Identify redundant or incoherent topics. Try a small validation-guided range of topic counts, then freeze the interpretation for downstream experiments.

Recommendation extension

Represent each movie by the mean topic distribution of its historical reviews. Form a user topic preference vector from rated movies. Compare topic-based similarity with genre-only ranking on held-out future interactions. Only use review information available before the cutoff; future reviews can leak knowledge into a historical recommendation experiment.

The small app does not use its offline LDA output for ranking. Its review-derived theme features are explicit dictionary matches, which are simpler and more interpretable for mid-sem.

33. Movie-specific named entity recognition

Named Entity Recognition, or NER, identifies spans referring to entities such as people, organizations and locations. In a movie application, you may want actor names, director names and movie titles. A general model's PERSON label does not tell you whether a person is an actor or director.

Two stages

Mention detection finds the text span. Entity linking connects it to a catalog identity. "Her" could be a movie title or a pronoun; "Jordan" could be a person's name or a location. Linking requires contextual evidence and a canonical catalog, not just capitalization.

A small extension design

Keep a catalog of movie titles, known aliases, actor names and director names. Use a general NER model and exact/normalized dictionary matching as candidate generators. Resolve ambiguous matches using nearby movie context. Return the original text span, entity type, linked ID if resolved and an uncertainty flag.

Evaluation data

Annotate start/end offsets, entity types and canonical IDs in real review/query text. Preserve original text so offsets stay valid. Include ambiguous titles, initials, spelling variants and entities absent from the catalog. Evaluate exact-span typed precision/recall/F1 and linking accuracy on correctly detected mentions.

Example failure

In "I liked Her more than the director's previous film," detecting "Her" requires context. In "I liked her performance," the same token should not become a movie entity. A title dictionary alone will overmatch.

Current scope

The small app does not ship a movie-specific NER model or entity-evaluation result. This is a final-stage extension with a concrete data contract and measurement plan. The presence of a general NER library in a notebook would demonstrate a tool, but would not establish movie-domain accuracy or complete the linking task.

34. Review summarization with supporting evidence

A movie-level review summary should preserve what people praise and criticize. A summary that says "Everyone loved the movie" after seeing only two positive reviews is unsupported, even if its grammar is excellent. Summarization quality depends on both source coverage and faithfulness.

Extractive summarization selects existing sentences or clauses. Abstractive summarization generates new wording. Extraction makes attribution easier but can be repetitive. Generation can be more readable but may introduce claims not supported by the reviews.

A simple final-stage pipeline

Group legitimate movie-linked reviews by movie ID. Deduplicate repeated text, detect aspects, and collect positive and negative evidence separately. Select a small set of representative clauses while limiting repeated content. Present aspect counts alongside the excerpts so the user can judge how much evidence exists.

A possible summary structure is: "Reviewers often praised the acting. Several criticized the pacing. Evidence: [review IDs and excerpts]." Replace "often" and "several" with counts or justified language based on actual data. One critical review should not be described as universal agreement.

Evaluation rubric

Rate faithfulness, coverage, balance and readability on a 1-5 scale using human annotators. Faithfulness asks whether each summary claim follows from the provided reviews. Coverage asks whether the important recurring points are represented. Balance asks whether significant praise and criticism both survive. Readability asks whether the summary is clear.

Automatic overlap metrics such as ROUGE can be useful supplementary signals when reference summaries exist, but overlap alone cannot prove factual support. A fluent unsupported sentence is still an error.

Current scope

The small frontend shows evidence for the currently typed review and uses candidate review signals for ranking. It does not provide a movie-level summarization screen. The evaluation utility accepts human summary-rubric scores collected from a later summary system; empty inputs produce no measured result rather than an artificial quality number.

35. Conversational QA: retrieve before answering

A movie question-answering system should connect answers to catalog facts and review evidence. "Which films have strong acting but slow pacing?" requires retrieval of relevant movie evidence, not merely a general language model's memory.

A minimal grounded design

Index catalog descriptions and review clauses with movie IDs, source IDs, text and dates. Convert a question into a query representation. Retrieve the most relevant records, apply explicit filters such as genre or excluded films, and construct an answer containing citations to those records. If evidence is insufficient, state that the corpus does not support the requested conclusion.

Lexical retrieval can begin with TF-IDF cosine similarity. Semantic retrieval uses dense embeddings that may match paraphrases better. A generation model can word the answer, but the retrieval results define the evidence boundary.

Retrieval and generation are separate

If the retriever misses the relevant review, even a strong language model lacks the right evidence. If retrieval succeeds but the answer changes the evidence's meaning, the generation stage fails. Measure both stages separately before combining them into one overall impression of quality.

Evaluation set

Create questions with known relevant movie/review IDs, expected factual points and unanswerable cases. Calculate Recall@k and MRR for retrieval. Have annotators check whether answers are correct, supported, complete enough and properly abstain when necessary.

Conversation state

Follow-up questions such as "only comedies" should update explicit filters while preserving the earlier user intent. Store selected constraints, not just a long concatenated transcript. Distinguish "I dislike horror" from a factual claim that every horror film is bad.

Mid-sem boundary

The supplied app is not a chatbot and makes no conversational retrieval quality claim. The handbook and evaluation format prepare you to add one later without confusing a catalog search demonstration with a validated question-answering system.

36. Recommendations: what history represents

A recommender selects candidate items that a user may value. CineScope starts with explicit ratings from the local browser profile. A high rating contributes positive genre preference; a low rating contributes negative preference. Already-rated films are excluded from the result set.

The history contains a movie ID, a rating and optional review text. It does not infer your identity from the MovieLens dataset. MovieLens community ratings provide a small item prior, while your browser history provides personal signals.

Signed rating weight

```
weight = (rating - 3) / 2
```

A rating of 5 gives +1, a rating of 3 gives zero, and a rating of 1 gives -1. A half-star rating gives -1.25. The scale is intentionally simple and asymmetric at the lowest half-star endpoint. This is a ranking design choice, not a learned utility scale.

Genre preference

For every historical film, add its weight to each of its genres. A candidate's genre component averages the accumulated preferences over its own genres, then divides by the total magnitude of historical rating weights, with a minimum denominator of one. This limits the influence of a long history and avoids division by zero.

Example

Suppose you rate a Comedy film 5 and a Horror film 1. Comedy receives +1 and Horror receives -1. With no other signals, an unseen Comedy candidate should outrank an unseen Horror candidate. If a film has both genres, its genre score combines both contributions.

What this method cannot infer

Genres are broad. Two Sci-Fi films can be completely different in tone, style and story. Sparse history makes the profile uncertain, and explicit ratings may not represent all reasons for liking a film. Review-derived theme and aspect signals add detail, but must be treated as noisy evidence rather than a complete personality model.

37. The exact hybrid ranking formula

The small recommender combines four components: genre affinity, review-theme match, review-aspect match and a damped community-rating prior. Its fixed weights make the logic easy to inspect. They have not been tuned or validated as optimal.

```
rank_score = 0.65 * genre_component
           + 0.15 * theme_component
           + 0.15 * aspect_component
           + 0.05 * rating_prior
```

Theme component

Historical review clauses with an explicit theme contribute a signed preference derived from their rule score. Candidate movies gain or lose affinity according to which themes appear in their available review text. The current candidate theme representation uses presence, not an average theme-polarity score, so it is a relevance signal rather than a quality guarantee.

Aspect component

Positive opinions about an aspect in your historical reviews create a preference proxy. Candidate review scores for those same aspects contribute signed quality signals. For example, praising acting in your reviews can favor a candidate whose review evidence praises acting. This proxy does not prove that acting is your most important decision criterion; it is a small interpretable approximation.

Rating prior

```
damped_mean = (rating_count * mean_rating + 20 * 3) / (rating_count + 20)
prior = (damped_mean - 3) / 2
```

Twenty imaginary neutral ratings shrink sparse averages toward 3. This is a simple smoothing prior, not a fitted Bayesian model. Fictional demo films have no fabricated community ratings, so their prior is zero.

Example calculation

For genre = 0.5, theme = 0.2, aspect = 0.4 and prior = 0.3, the score is 0.325 + 0.03 + 0.06 + 0.015 = 0.43. Its purpose is ordering. It is not a star prediction or calibrated enjoyment probability. The UI exposes components and textual reasons so the result is explainable.

38. Connecting real reviews to recommendation features

The mid-sem engine already connects review analysis to ranking through theme and aspect features. On the fictional catalog, candidate review text is synthetic and visibly labeled. On a MovieLens catalog, those features remain absent until you attach legitimate movie-linked reviews.

Required data contract

Provide `movieId,review` CSV rows to the importer. Movie IDs must use the MovieLens namespace. Do not join different sources using review filenames or ambiguous title strings. Keep provenance and permissions in your research data folder even though the small serving catalog stores only simplified fields.

Four extension options

- Explicit aspect vectors: six aspect means plus counts and missingness flags.
- Topic vectors: movie distributions over learned corpus topics.
- Word/document embeddings: dense semantic representations of review content.
- Evidence-aware text features: a bounded vocabulary of meaningful phrases and themes.

Compare one addition at a time against a genre-only baseline. A larger feature vector does not guarantee better ranking, especially when review coverage is sparse or biased toward popular films.

Missing information

A candidate with no review should not receive invented average aspect scores. Keep a missingness indicator or use a fallback ranking component. Report feature coverage: the proportion of candidate movies and recommendation requests for which review-derived features are actually available.

Temporal leakage

If you evaluate recommendations as of January, do not include a movie review published in March. Likewise, a user's test-period rating should not enter the history used to predict that same interaction. Align metadata, review timestamps and interaction cutoffs as far as the dataset permits, and disclose any unavailable timestamps.

Ablation study

Measure genre only; genre plus themes; genre plus aspects; genre plus both. Keep the user split and candidate set fixed. Report ranking metrics and coverage. This study directly tests whether review analysis helps recommendation, instead of assuming a benefit because the components are connected in code.

39. Cold start, fallback and recommendation limits

Cold start means that useful history is unavailable. A new user has no ratings. A new movie may have no ratings or reviews. A content-based system can still use movie genres, but it cannot manufacture personal preference or review evidence.

What the prototype does

For a new user, genre, theme and aspect preference components are zero. With a real MovieLens catalog, the small rating prior orders candidates. With the fictional catalog, all priors are zero, so titles break ties alphabetically. The interface calls these catalog fallbacks rather than pretending they are deeply personalized.

After a few ratings, the profile becomes more specific. A reviewed film is excluded even if its rating is neutral. Removing a history entry makes that film eligible again. Unknown IDs from a different catalog are ignored.

Why recommendations can look repetitive

Many highly ranked films may share one favored genre. A final extension can add diversity by penalizing candidates too similar to movies already selected in the list. Diversity trades some immediate similarity for broader coverage. Evaluate user satisfaction or a defined diversity metric rather than claiming that more variety is always better.

Popularity and selection bias

Community ratings come from people who chose to watch and rate films. An unobserved interaction is not necessarily dislike. Popular films receive more evidence, while niche films may be underrepresented. The bounded Spark UI export further favors highly rated-count movies, which is acceptable for a small serving demonstration but changes the candidate population.

Minimum honest explanation

Tell the user which liked genres or review signals contributed. Do not say "people like you loved this" unless collaborative data actually supports that claim. The current system is an explainable content/history hybrid; it is not an ALS collaborative-filtering implementation.

40. Evaluating recommendations properly

A ranking system should be evaluated on future or held-out interactions. A useful protocol orders each user's ratings by timestamp, trains on earlier interactions, uses a later validation block for choices and reserves the final block for testing. Report how users with very short histories are handled.

Define relevance before evaluating, for example a held-out rating of at least four stars. Define the candidate set, exclude training-seen items, and state whether evaluation considers all available movies or sampled negatives. Metrics from different candidate policies are not directly comparable.

Core ranking metrics

Precision@k is the fraction of the k suggested positions that are relevant. Recall@k is the fraction of relevant held-out items that appear in those positions. MRR rewards how early the first relevant result appears. NDCG discounts relevant items at lower ranks and normalizes against an ideal ordering.

```
DCG@k = sum(relevance_at_rank_i / log2(i + 1)), with ranks starting at 1
NDCG@k = DCG@k / ideal_DCG@k
```

Tiny example

With two relevant movies A and C, a top-three list [B, A, C] has Precision@3 = 2/3 and Recall@3 = 1. Its first relevant result is at rank two, so reciprocal rank is 1/2. NDCG is below one because placing A and C first would be better.

Baselines and coverage

Compare a popularity baseline, genre-history baseline and the hybrid. Report catalog coverage, user coverage and the proportion of items with usable review features. A model that cannot score many cold-start items may look good only because hard cases were silently dropped.

Current utility

`evaluate_pipeline.py` computes binary-relevance ranking metrics from supplied ranked and relevant ID lists. It skips queries with no relevant items and reports the count skipped. It does not create a valid split for you. The evaluation protocol remains your responsibility, and no real recommendation-quality result is bundled.

41. Big data concepts before Spark syntax

Big data is not defined by a project title. It refers to workloads whose size, speed or complexity challenge the available conventional processing setup. A small CSV on one laptop may be best handled by ordinary Python, even if the same computation can later be distributed.

Parallel processing divides work into pieces that can execute simultaneously. Distributed processing places those pieces on multiple machines. Spark provides a programming model for distributing data operations and recovering work when tasks fail. Running Spark in local mode still uses one machine.

MovieLens operations that scale naturally

Reading partitioned ratings, filtering invalid rows, grouping by movie ID, calculating counts and averages, joining movie metadata, and writing columnar output are suitable batch operations. Their output can be much smaller than the raw input and can be exported to a serving application.

The hidden cost: shuffles

A groupBy often needs records with the same key to meet on the same worker. Moving records across partitions is called a shuffle. Joins and sorting can also trigger shuffles. Network transfer, serialization and skew can dominate the runtime even if each arithmetic operation is cheap.

Skew

If one film has vastly more ratings than others, its key can create uneven work. Inspect partition sizes and task duration distributions. Techniques such as preaggregation, appropriate partitioning or special handling of heavy keys may help, but should address observed bottlenecks.

What to claim in mid-sem

"The Scala job uses distributed DataFrame operations and writes scalable storage output" is defensible from the source. "Spark is faster than Python" requires a controlled measurement. Small inputs often favor Python because Spark startup and scheduling overhead exceed the work being parallelized.

42. Scala essentials for this project

Scala runs on the JVM and combines object-oriented and functional programming features. For this project, you need only a small subset: objects, methods, immutable values, method chaining and imported library functions.

`object MovieStats` defines a singleton entry point. `def main(args: Array[String]): Unit` is the method called when the program starts. `args(0)` is the first input argument and `args(1)` the output

directory. `require` stops the program if the argument count is wrong.

Values and chaining

`val ratings = ...` binds a value that is not reassigned. The object represented by that value is a Spark DataFrame. A chain such as `.na.drop(...).filter(...).dropDuplicates(...)` describes successive transformations of the dataset.

Columns are expressions

In Spark, `col("rating")` refers to a distributed column expression, not one local rating value. `avg("rating")` describes an aggregate operation. You do not normally loop over every row on the driver to calculate the total manually.

Why types matter

CSV text must become typed columns. A rating is a Double, IDs are Integers and the Unix timestamp is a Long. Explicit schemas avoid relying on guesses and make the data contract visible. The Scala source includes two schemas: one for ratings and one for movie metadata.

Build files

`build.sbt` specifies Scala and Spark dependencies. Spark is marked "provided" because `spark-submit` supplies its runtime libraries. The Scala binary version used to compile the artifact must match the Spark distribution's binary version. The target package uses Scala 2.12 with Spark 3.5.7; it is not interchangeable with a Scala 2.13 build.

Viva explanation

"Scala describes the batch transformations; Spark executes them across partitions. My program reads typed CSV data, cleans rows, groups ratings by movie, joins metadata and writes reusable output." This explains the role without needing to memorise unrelated Scala language features.

43. Walk through MovieStats.scala

The Spark program has one job: convert raw MovieLens ratings into reusable statistics and a small frontend catalog. It does not train sentiment, ALS or a streaming model. Its focused scope keeps the mid-sem code readable.

Read and clean

A SparkSession creates the application context. Ratings and movies are read using explicit schemas and header handling. Rating rows missing required values are removed, values outside 0.5-5 are filtered, and exact rating-event duplicates are dropped. This is a basic cleaning policy; it does not claim to resolve every corrupted or conflicting user-item record.

Movie metadata are deduplicated by movie ID. If your source can contain conflicting duplicate metadata, audit and resolve those conflicts deterministically before this step. The source dataset's ID uniqueness is an assumption worth validating.

Aggregate and join

The ratings groupBy calculates `rating_count` and `mean_rating` for each movie. A left join attaches them to movies. Movies with no ratings receive count zero and the neutral prior mean of three for serving convenience. That filled value is not a measured community average.

Genre aggregation

The pipe-separated genre field is split and exploded so a multi-genre movie contributes to each listed genre. Genre mean is weighted by movie rating counts. An Action/Comedy movie's rating events contribute to both groups, so genre counts overlap and must not be summed as unique global ratings.

Write outputs

Full joined movie statistics are written as Parquet. Genre statistics are written as CSV. A deterministic top-2,000 selection by rating count, then movie ID, becomes the bounded JSON export for the frontend.

Only that bounded export is coalesced to one partition. The program does not collect all raw ratings into driver memory. All outputs use error-if-exists behavior, so a new run should use a new output folder.

44. Build, run and connect the Scala output

Use Java 17 and a Spark 3.5.7 distribution built for Scala 2.12, matching the supplied Scala 2.12.18 build. The chosen versions preserve compatibility with the original project's stack; this is not a claim that they are the newest available versions. Refer to the matching Spark documentation when installing. [R7]

```
cd scala
sbt package
cd ..
spark-submit --master "local[2]" --class MovieStats \
  scala/target/scala-2.12/cinescope-midsem_2.12-1.0.0.jar \
  /path/to/ml-latest-small output/run1
```

The backslash continuation is for Unix-style shells; Windows users can place the command on one line or use WSL. Use a fresh output folder on every run. `local[2]` requests two local worker threads; it does not create a two-machine cluster.

Output interpretation

Spark writes directories containing part files, not necessarily one ordinary file with the requested directory name. Parquet data are under `movie_stats`, CSV part files under `genre_stats` and JSON-lines part files under `ui_catalog`.

```
python scala/import_spark_catalog.py output/run1/ui_catalog
```

This converts the bounded JSON-lines output to the JSON array expected by the application. Then set `CINESCOPE_CATALOG=data/movielens.json` before restarting the app.

Troubleshooting

A Scala binary mismatch can produce linkage or missing-method errors even when Java is installed. Missing Spark classes during execution may mean the JAR was run directly with `java` instead of `spark-submit`. A path that works on the driver may not be readable by cluster workers; use shared storage for a distributed deployment.

Validation status

The delivery environment did not have sbt or Spark installed, so this Scala job was not compiled or executed here. Its source is provided for your configured environment. Run it on MovieLens-small first, inspect row counts and weighted means, and only then scale the input.

45. From local batch to a genuine scale experiment

Moving from MovieLens-small to a larger rating dataset should change the workload, not the meaning of the computation. Retain explicit schemas, output definitions and duplicate policies. Start by confirming correctness on a small sample where you can calculate expected aggregates independently.

Benchmark design

Record input bytes, input row counts, number of output movies, hardware, core count, memory limits, Spark version and partition configuration. Measure end-to-end wall time including the stages you intend to compare. State whether JVM startup and cold file-cache effects are included.

Run repeated measurements under comparable conditions. Report a median and spread rather than selecting the fastest run. If comparing Python and Spark, make both methods read equivalent input and calculate equivalent output. Comparing a warm in-memory operation with a cold distributed read is misleading.

Scaling quantities

Speedup is T1/Tp, where T1 is runtime under the reference configuration and Tp is runtime with p workers or cores. Parallel efficiency is speedup/p. Neither must increase indefinitely: scheduling, shuffling and serial work limit scaling.

Storage choices

CSV is readable but repeatedly parsing it is expensive. Parquet stores typed columns and can support efficient scans of selected fields. Partition outputs according to realistic query patterns and avoid producing huge numbers of tiny files. A storage format change can matter as much as adding workers.

What to inspect in Spark

Use the physical plan and Spark UI to identify shuffles, task skew, spill, executor memory pressure and stages with little parallelism. Do not add caching everywhere: cache only reused datasets when the memory cost is justified.

A result worth presenting

Even if Spark is slower on the smallest input, explain why its overhead dominates there and whether larger inputs change the balance. A careful negative result is more useful than an unsupported speedup claim.

46. ALS and streaming: keep them as final-stage extensions

The uploaded final project included more advanced components. The simplified mid-sem build intentionally focuses on content/history ranking and batch aggregation. Two useful future extensions are collaborative filtering with ALS and review-event streaming.

ALS intuition

Alternating Least Squares approximates a user-item rating matrix using a low-dimensional vector for each user and each item. Their dot product predicts a rating. Training alternates between solving user factors and item factors while regularizing large values.

ALS uses patterns across many users, unlike the current single-profile genre recommender. It also has cold-start limits: a new user or new item lacks learned factors unless you add a fold-in or fallback strategy. An explicit-rating ALS objective does not directly optimize top-k recommendation quality.

Evaluation boundary

Report RMSE or MAE for rating prediction and ranking metrics for recommendation lists. Also report the fraction of test interactions the model can score. Silently dropping unseen users/items can inflate the apparent result. Compare against simple means and popularity baselines.

Streaming intuition

A streaming pipeline processes events as they arrive, such as newly submitted reviews. A robust event needs an ID, timestamp, source and payload. Checkpoints remember progress; deduplication rules decide how repeated events are handled; late-data policies decide what happens to old events arriving after newer ones.

Typing into the current browser form is interactive request/response inference, not a distributed streaming system. There is no Kafka or Structured Streaming process behind the UI. If you add one later, benchmark ingestion delay and fault recovery separately from the NLP model's classification quality.

Why defer these features?

They introduce new data contracts, evaluation protocols and operating requirements. First make the small system understandable and correct. Then add a single advanced component, prove its value on measured data and retain the working baseline for comparison.

47. Classification metrics and confusion matrices

A confusion matrix counts gold labels against predicted labels. For binary sentiment, designate positive as the target class and count true positives, false positives, false negatives and true negatives. The same underlying predictions support several metrics, each answering a different question.

```
precision = TP / (TP + FP)
recall = TP / (TP + FN)
F1 = 2 * precision * recall / (precision + recall)
accuracy = correct_predictions / all_predictions
```

Worked example

Suppose 20 truly positive reviews include 16 predicted positive and four predicted negative. Of 20 truly negative reviews, five are predicted positive and 15 negative. Positive precision is 16/21, about 0.762. Positive recall is 16/20 = 0.8. Positive F1 is about 0.780. Accuracy is 31/40 = 0.775.

The negative class has its own precision, recall and F1. Macro-F1 averages class F1 values equally, so a smaller class is not automatically overwhelmed by a larger one. Weighted F1 weights classes by support. Always report the averaging convention.

Why accuracy can mislead

If 95% of examples are positive, always predicting positive achieves 95% accuracy while completely failing to identify negative reviews. A class distribution table and confusion matrix make that failure visible. Include a majority baseline in your experiment report.

Aspect metrics

Measure each aspect separately and show support for positive, negative, neutral, mixed and not_mentioned labels. A large number of not_mentioned examples can dominate accuracy. Consider reporting mention detection separately and sentiment metrics conditional on true mentions, while clearly stating both denominators.

Evidence quality

Correct aspect polarity with the wrong evidence span is still a partial failure. For a richer annotated dataset, calculate exact-span match or token-overlap precision/recall. A claim of explainability should be checked against whether the quoted text genuinely supports the score.

48. Evaluating the complete NLP pipeline

A complete evaluation should cover each component and the end-to-end user task. Sentiment accuracy alone cannot establish reliable aspect scores, correct entity linking, faithful summaries or useful recommendations.

The optional `evaluate_pipeline.py` utility reads a JSON file containing gold/predicted labels, typed entity spans, ranked retrieval IDs, recommendation IDs and human summary ratings. It computes metrics without inventing missing labels. Start from `data/evaluation_template.json` and fill only tasks you actually evaluated.

Input examples: schema illustrations, not benchmark results

```
{
  "sentiment": [{"gold":"positive","pred":"negative"}],
  "aspects": {"Acting":[{"gold":"positive","pred":"positive"}]},
  "ner": [{"gold":[[0,4,"TITLE"]],"pred":[[0,4,"TITLE"]]}],
  "retrieval": [{"relevant":["r7"],"ranked":["r2","r7"]}],
  "recommendations": [{"relevant":["ml:1"],"ranked":["ml:2","ml:1"]}],
  "summaries": [{"faithfulness":4,"coverage":3,"balance":4,"readability":5}],
  "k": 10
}
```

For NER, spans use start, end and type in the same original text, with end exclusive. The evaluator uses exact typed span matching. For summaries, each dimension uses a 1-5 human rubric; the utility averages available ratings and reports their count. Empty tasks return null metrics or zero evaluated counts, not perfect scores.

Important limits

The utility does not collect human judgments, generate summaries, retrieve documents or create valid data splits. It measures supplied outputs. It also does not compute entity-linking accuracy or answer faithfulness automatically; those require additional gold fields or human assessment.

End-to-end assessment

Ask evaluators to complete tasks such as finding a movie with strong acting and saving a preference. Record whether the evidence was correct, whether the recommendation was useful, and whether the interface was understandable. Track model errors separately from usability failures so improvements target the right component.

49. Calibration, uncertainty and error analysis

A model can classify well while expressing probabilities that are too confident. Calibration asks whether events predicted near probability p actually occur about p of the time across many comparable examples. A rule score based on sentiment word counts has no such probabilistic guarantee.

A simple diagnostic

Group held-out predictions into probability bins, compare mean predicted positive probability with the observed positive fraction, and report sample counts. Small bins are noisy. If you calibrate probabilities, use validation data or a dedicated calibration split, not the final test labels.

Uncertainty is broader than one number

A short review may lack evidence. A review may contain sarcasm or a language the tokenizer does not support. A long review may be truncated by the transformer. A new movie may have no reviews. These are different uncertainty sources and should not be collapsed into one confident-looking percentage.

Error buckets

Manually tag a sample of errors into categories: negation, sarcasm, implicit aspects, mixed sentiment, named-entity ambiguity, spelling, multilingual text, long-document truncation and dataset-label ambiguity. Count each category and keep representative examples with IDs.

Paired comparisons

Because the baseline and transformer prediction files retain review IDs, you can inspect examples where only one model is correct. For statistical uncertainty, bootstrap at an appropriate independent unit, such as movie group when reviews are correlated within films. A tiny difference in macro-F1 may not be meaningful on a small test set.

What to improve first

Choose a frequent, consequential error category and propose a targeted intervention. Re-evaluate on the same held-out protocol plus a separately collected challenge set. Do not endlessly modify rules to fit a handful of test examples and then present those same examples as an unbiased evaluation.

50. Reproducibility and reporting actual results

A reproducible experiment includes code, data provenance, split IDs, configuration, environment, model artifact and measurement output. A notebook screenshot or a remembered accuracy value is not enough to reconstruct what happened.

Save a run record

Record the dataset source and checksum, cleaning policy, deduplication counts, random seeds, label mapping, feature settings, optimizer settings, hardware and package versions. Keep prediction IDs and raw metrics. State which code version generated the result.

Training versus testing status

"Script included" means code exists. "Smoke-tested" means a small controlled example checked its behavior. "Trained" means parameters were actually fit on a named dataset. "Evaluated" means predictions were compared with defined held-out labels. "Validated in the UI" means the serving path was exercised. These stages should not be merged into a single word such as completed.

The new delivery includes core behavior tests, HTTP checks, numerical inference parity and small importer/evaluator checks. Real-corpus training, heavy embedding/transformer runs and Scala compilation are not claimed. Consult `docs/VALIDATION.md` for the final execution record.

Environment files

The optional requirements define broad compatible major-version ranges rather than a tested exhaustive lockfile. After successfully running experiments in your environment, save `python -m pip freeze` and your Java/Spark versions. Record the exact pretrained model revision if you need immutable model downloads.

Honest tables

Use blank or "not run" cells until an experiment produces output. Do not copy metrics from the original large project into this simplified implementation. Changes in cleaning, data splits, features or evaluation candidates make those earlier values non-transferable.

A useful final conclusion

Explain which method worked best under your protocol, which errors remain, how much computation it required and where the result may not generalize. A well-supported limited conclusion is stronger than declaring that one model understands every movie review.

51. Advanced foundations: loss, attention and representation

Once the baseline is understandable, deeper mathematics helps explain what larger models actually optimize. It also exposes why a model can succeed on one metric and fail on another.

Logistic loss

For label y in {0,1} and predicted positive probability p, binary cross-entropy is `-[ylog(p) + (1-y)log(1-p)]`. A correct but uncertain prediction receives some loss; a confident wrong prediction receives a large loss. Regularization adds a penalty on model weights to reduce overfitting.

For a linear logit z = w dot x + b, the gradient of the unregularized loss with respect to w is `(p-y)*x`. If the model is too positive on a negative example, the gradient pushes weights away from the active positive evidence. Optimization repeats this over many examples.

Attention intuition

A transformer constructs query, key and value vectors for tokens. Attention compares queries with keys, normalizes the resulting scores and combines value vectors. The standard scaled dot-product expression is `softmax(QK^T / sqrt(d_k)) V`. Context changes which information contributes to each token representation.

This flexibility can help associate distant words, but attention weights are not automatically a faithful explanation of a prediction. Demonstrating reliable explanations requires separate evidence or intervention-based evaluation.

Representation versus objective

Word2Vec learns word vectors from context prediction. A sentiment model learns a decision related to polarity labels. An LDA model describes document-topic mixtures. An ALS model approximates interaction structure. They optimize different targets, so a representation that looks semantically appealing may not improve your actual recommendation metric.

Expert habit

For every model, write four lines: input, representation, objective and evaluation. Then ask whether the evaluation matches the user-facing goal. This habit prevents using low language-model perplexity as proof of sentiment quality, or low rating RMSE as proof of top-k recommendation usefulness.

52. The full 15-topic syllabus map

This table distinguishes what runs now from what requires further research. It replaces vague completion claims with observable deliverables.

Topic	Delivered here	Evidence still needed for final stage
1. Preprocessing	Core cleaning, tokenization, clauses, negation; examples	Broader language and spelling challenge evaluation
2. Morphology	Detailed controlled-comparison plan	Run stemming/lemmatization ablation
3. Syntax/semantics	Nearest-aspect rules; dependency integration design	Integrated parser and gold target evaluation
4. Collocations	Real-corpus PMI script	Execute on downloaded corpus and interpret output
5. One-hot/BoW/TF-IDF	Explanations; controlled BoW/TF-IDF training	Real-data metrics and experiment environment
6. N-gram LM	Executable bigram/trigram counts, smoothing, perplexity, notebook	Real-corpus run and smoothing comparison
7. Embeddings	CBOW/Skip-gram and optional GloVe script	Training, similarity assessment and coverage
8. Sentiment	Rules; optional LR and DistilBERT experiments	Same-split measured comparison
9. Aspect sentiment	Working rules; human template; six-classifier script	Human annotations and per-aspect test reports
10. Topic modelling	Real-training-corpus LDA script	Execute, label topics, assess coherence/stability
11. NER	Domain design and span metric utility	Movie-domain detector/linker and gold annotations
12. Summaries	Evidence display; rubric evaluation support	Movie-level summary generator and human assessment
13. Conversational QA	Retrieval/answer design and ranking evaluator	Retrieval service, grounded answers, QA benchmark
14. Text recommendations	Review themes/aspects integrated into ranker	Real linked reviews and ablation quality results
15. Evaluation/visualization	Score bars, component explanations, reports and metric utility	Real benchmark plots, human studies and scale runs

How to use this table

For mid-sem, demonstrate the working rows and explain the remaining steps. For final submission, replace each planned evidence cell with a dataset, run identifier and result artifact. Do not change a cell to completed merely because you installed a library or copied a sample notebook.

53. The six priorities as an execution plan

The requested additions can be completed without turning the mid-sem app into an oversized application. Keep experiments in `optional/` and integrate only the pieces that demonstrate measured value.

Priority 1: Word2Vec and GloVe

Prepare the shared review corpus. Train CBOW and Skip-gram, then compare the same query words with local pretrained GloVe vectors. Deliver saved vectors, coverage counts, neighbor tables and a short interpretation. A stronger follow-up is downstream sentiment or recommendation evaluation.

Priority 2: a genuine n-gram model

Run the notebook toy example to understand the formula, then run the real corpus script. Report bigram and trigram perplexity with the same vocabulary policy. Tune smoothing on validation data if extending the experiment.

Priority 3: train DistilBERT

First obtain the BoW/TF-IDF report. Fine-tune DistilBERT using the same CSV split. Preserve the truncation setting and hardware details. Compare quality, latency and resource cost; retain paired prediction IDs.

Priority 4: train aspects

Write an annotation guide, label a pilot independently, resolve disagreements and expand the dataset. Run the six-classifier baseline. Report every aspect and label with support. Do not derive gold aspect sentiment from a whole-review label.

Priority 5: use review information in ranking

The prototype already supports explicit theme/aspect signals. Attach real movie-linked reviews, measure their coverage, and compare genre-only ranking with the review-enhanced variants. Add embeddings or topics only as separate, measured extensions.

Priority 6: evaluate the whole pipeline

Collect task-specific gold data for entities and retrieval, and human judgments for summaries. Use the provided evaluator where its schema fits, adding explicit measures for linking and generated-answer support. Produce a result dashboard that keeps absent evidence distinct from a zero score.

Dependency order

Reliable IDs and shared splits come first. They enable fair comparisons and valid joins. Human aspect labels come before aspect training. Grounded retrieval comes before a generative conversation layer. These dependencies matter more than adding every library as quickly as possible.

54. A practical semester schedule

The following sequence is a suggested order of work, not a promise of fixed training times. Adjust the number of sessions to your coursework and hardware. Each stage has a concrete exit condition.

Stage	Main work	Exit condition
1	Run and understand core app	Explain every visible output and null value

Stage	Main work	Exit condition
2	Test English review examples	Save a small behavior table including failures
3	Download/import MovieLens	Real titles appear; history ranking updates
4	Run Scala on small ratings	Inspect Parquet and weighted genre aggregates
5	Prepare IMDb and baselines	Save split manifest and held-out report
6	N-gram, PMI, LDA, Word2Vec	Real-corpus experiment outputs with interpretation
7	Human aspect annotation	Clear guide, split IDs, label support and agreement
8	DistilBERT and aspect training	Measured comparison and error analysis
9	Real review-aware ranking	Coverage and ablation results
10	Summaries, entities, retrieval QA	Gold/rubric evaluation and grounded examples
11	Scale and robustness	Runtime benchmark and documented limitations
12	Final report and demo	Reproducible results, screenshots and concise conclusion

Mid-sem stopping point

A sensible checkpoint is stage four, with a small optional overall-sentiment training result if time allows. You can demonstrate an integrated system without pretending that all advanced experiments are finished. The remainder becomes an explicit final-stage roadmap.

When to simplify further

If the interface works but training is blocked, keep the rule engine and explain it. If Spark setup is blocked, run the Python importer and present the Scala implementation with an honest unexecuted status. Do not spend the entire project adding infrastructure while the NLP data and evaluation remain unclear.

When to expand

Expand after an observed limitation gives you a reason. Add dependencies when proximity errors are common; add a transformer when the baseline's context failures matter; add distributed execution when measured data volume justifies it. This connects each feature to a problem you can demonstrate.

55. A five-minute mid-sem demonstration

Begin with the objective: "Our prototype breaks a movie review into overall sentiment and aspect opinions, remembers a local user's ratings, and recommends unseen films. Scala/Spark provides an offline rating-analytics path."

Minute 1: live review analysis

Choose a fictional demo film and click the example. Point out positive Acting, negative Story and positive Music. Explain that the overall score can be balanced while individual aspects disagree. Expand one evidence clause.

Minute 2: missing evidence and negation

Type "The story was not bad." Explain the local negation flip. Then type a factual sentence with no evaluative words and show the missing score. State that no evidence is different from neutral sentiment and that sarcasm remains difficult.

Minute 3: history and recommendation

Save a rating and a review. Show that the film disappears from the recommendation list and that reasons reflect liked genres or review signals. Save a low rating for another film and inspect changes. Refresh to demonstrate browser-local persistence.

Minute 4: data and Scala

Show the MovieLens importer or the prepared real catalog. Explain that MovieLens contains ratings and genres, while IMDb supplies overall sentiment labels. Walk through the short Scala sequence: read, clean, group, join and write. Show actual output only if you have run it.

Minute 5: limitations and next stage

Explain the rule-based baseline, the optional TF-IDF training path and the six advanced priorities. End with concrete next steps: human aspect labels, same-split model comparisons, linked real reviews and complete-pipeline evaluation.

A suitable spoken conclusion

"The mid-sem prototype demonstrates the complete interaction from review input to explainable recommendation. Its current analysis is intentionally simple and transparent. The final stage will replace selected rules with evaluated models and test quality and scale using real datasets."

This wording accurately describes implementation progress without claiming that a planned experiment has already succeeded.

56. Viva bank: beginner and implementation questions

1. What problem does CineScope solve?

It separates a movie review into different opinions and uses a user's rating history to recommend unseen films. A single star rating does not show whether the person liked acting, music or story.

2. What is sentiment analysis?

It identifies evaluative polarity in text, such as praise or criticism. It does not measure a movie's objective quality.

3. What is an aspect?

An aspect is a component being discussed, such as Acting or Ending. Different aspects in one review can receive different sentiments.

4. Why do some scores show no evidence?

The engine did not find a supported local opinion for that aspect. Assigning zero would incorrectly imply negative sentiment.

5. What does 100 mean in the default app?

All detected signed opinion hits in that calculation were positive. It is a heuristic index, not 100% accuracy or confidence.

6. How does live analysis work?

The frontend waits 300 ms after typing pauses, sends text to the backend and renders JSON results. A request counter prevents outdated analysis responses from overwriting newer input.

7. Why Python for the backend?

A short standard-library server keeps the app easy to run and inspect. Scala is reserved for the requested big-data batch component.

8. Why not use only MovieLens for NLP?

MovieLens provides recommendation ratings and metadata, not the required labeled full-review sentiment corpus. Overall sentiment and aspect training need separate labels.

9. Is the demo catalog real?

No. Its titles and reviews are authored demonstration data, clearly labeled. A MovieLens importer is supplied for real metadata and ratings.

10. How is history stored?

In browser localStorage, with one latest entry per movie and a maximum of 200 movies. The server does not maintain user accounts.

11. How does the engine handle mixed opinions?

It splits simple clauses and assigns nearby opinion words to aspect keywords. Overall sentiment combines the opinion hits, while aspect outputs remain separate.

12. How is negation handled?

A small window checks for negation tokens before an opinion word and flips its sign. This is limited and can fail on complex constructions.

13. Do you remove all stop words?

No. Negation words must be preserved for sentiment. The topic-modelling experiment uses a different preprocessing choice because it has a different objective.

14. What is the difference between a theme and genre?

A theme is an idea such as friendship; a genre is a category such as Comedy. An aspect is a production or storytelling component such as acting.

15. What does theme coverage mean?

It is the fraction of split review clauses that explicitly mention that theme. It is not theme probability or overall movie quality.

16. Why exclude seen films?

The demonstrator's recommendation task is to suggest unseen options. Rating a film marks it as seen for this local profile.

17. How do dislikes affect recommendations?

Low ratings create negative genre contributions, lowering similar candidates relative to other options. This remains a simple preference approximation.

18. Is this collaborative filtering?

The small live recommender is content/history based. It does not train user-item latent factors or find similar MovieLens users.

19. What is the main limitation?

Rules miss context, implicit targets, sarcasm and unfamiliar language. Recommendation quality is also unmeasured until a valid held-out experiment is run.

20. What did you simplify from the final project?

The mid-sem version removes large service layers, extra screens and automatic heavy-model paths. It keeps the review-to-recommendation interaction and a focused Scala job.

57. Viva bank: training, big data and research questions

21. What is BoW?

A count vector over a fixed training vocabulary. It treats a document as feature counts and needs explicit n-grams to retain limited local order.

22. How does TF-IDF differ?

It reweights counts using training document frequency, then the chosen implementation normalizes the vector. Rare terms receive more relative weight.

23. Why logistic regression?

It is a compact learned baseline with understandable weights and relatively modest training cost. It provides a reference before introducing transformers.

24. What is data leakage?

It occurs when training or model selection uses information that would not be available at prediction time, such as test labels or future user interactions.

25. Why a validation set?

It supports parameter and checkpoint selection while preserving a separate test set for final evaluation.

26. Why not report only accuracy?

Class imbalance can make accuracy look strong while a minority class fails. Macro-F1, class support and confusion matrices expose more behavior.

27. Why compare models on identical test IDs?

Different test examples can change difficulty. Shared IDs make results and error comparisons more interpretable.

28. Is a bigram TF-IDF feature a language model?

No. A language model normalizes conditional token probabilities and can be evaluated by perplexity. A classifier uses bigram features to predict labels.

29. Why smoothing in an n-gram model?

It avoids zero probability for unseen sequences. Otherwise a single unseen continuation can make test sentence probability zero.

30. Can a trigram model be worse than a bigram?

Yes. Longer contexts are sparser, so probability estimates may be worse on limited data.

31. What is CBOW versus Skip-gram?

CBOW predicts a word from context; Skip-gram predicts context from a word. The supplied experiment trains both on the same training corpus.

32. Why can good and bad be near in word-vector space?

They often occur in similar contexts. Distributional similarity is not identical to sentiment agreement.

33. What does DistilBERT add?

Contextual representations and pretrained language information. Its benefits must be measured against its compute and serving cost.

34. Why can't overall labels train all aspects?

One review can praise acting and criticize story. Copying the overall label to every aspect would introduce false labels.

35. How will you evaluate NER?

Compare predicted spans and types with human annotations, and evaluate linking to movie/person IDs separately.

36. How will you evaluate summaries?

Use human checks for faithfulness, coverage, balance and readability, with overlap metrics only as supplementary evidence when references exist.

37. What is a Spark DataFrame?

A distributed table with named, typed columns and an optimized execution plan for relational operations.

38. What is a shuffle?

Redistribution of data across partitions so matching keys can be grouped or joined. It can be expensive in network and disk work.

39. Why avoid collecting all data?

Collecting sends the full result to driver memory, removing the benefit of distributed storage and risking memory exhaustion.

40. Why coalesce the small UI export?

Only a bounded 2,000-row selection is made convenient for local serving. The full statistics remain distributed in Parquet.

41. Does local[2] prove cluster scalability?

No. It uses two local worker threads. Cluster scaling requires measurements on a defined multi-worker setup.

42. Why can Spark be slower on small data?

Startup, scheduling and shuffle overhead can exceed the cost of the actual aggregation. Measure rather than assume a speedup.

43. Why does low ALS RMSE not guarantee good recommendations?

Rating error and top-k ranking quality are different objectives. A model can approximate ratings without putting the most useful items first.

44. What is NDCG?

A ranking measure that rewards relevant items near the top and normalizes by the ideal order for the same relevance set.

45. What is cold start?

A new user or item lacks historical signals. The prototype uses a fallback and content features rather than inventing history.

46. How will you prove review features help?

Run ablations with fixed splits and candidates: genre only, plus themes, plus aspects, and both. Report ranking metrics and feature coverage.

47. What is calibration?

Agreement between predicted probabilities and observed frequencies over many labeled examples. A rule score is not inherently calibrated.

48. Which advanced results are already measured?

None of the new real-corpus heavy training or large-scale benchmarks are claimed. The delivery records functional tests separately from research evaluation.

49. What should final-stage work prioritize?

Reliable labels and IDs, controlled model comparisons, real review integration, component-specific evaluation and scale measurements.

50. What makes the project scientifically credible?

Clear scope, reproducible data splits, strong simple baselines, honest limitations and results supported by saved predictions and evaluation protocols.

58. Troubleshooting and common misconceptions

Symptom	Likely cause	Action
Browser cannot reach app	Server stopped or wrong port	Keep terminal open; verify printed URL
Page opens but API fails	HTML opened as a local file	Launch app.py and use its HTTP address
Scores look extreme	Small lexicon uses signed hit ratio	Explain index semantics; train and evaluate baseline
Movie genre absent from review tags	Metadata and review mentions differ	Read metadata separately from text evidence
New model is not active	Server loaded old state	Stop and restart after creating sentiment.json
Aspect result did not change after training	Overall model only is served	Train/integrate aspect models separately
MovieLens import cannot find files	Wrong extracted directory	Point to folder containing movies.csv and ratings.csv
Spark output already exists	Deliberate error-if-exists mode	Use a new run directory
Spark class or method error	Version or launch mismatch	Align Scala/Spark and use spark-submit
Word2Vec query is missing	Below min_count or unseen	Report unknown coverage; do not invent neighbors
Transformer training is very slow	CPU or long dataset	Start with a documented fixed pilot subset
Summary/NER metrics are null	No labeled evaluation rows	Collect gold/rubric data first

Misconception: a high score proves the model is correct

A prediction can be confident and wrong. Accuracy and F1 are calculated over labeled datasets. A single example demonstrates behavior, not statistical reliability.

Misconception: more advanced always means better

A transformer trained on unsuitable data can underperform a simple baseline. A distributed job can be slower on tiny inputs. More components can increase failure points. Let measured results determine which complexity is justified.

Misconception: all unknowns should become neutral

Unknown means the system lacks evidence. Neutral means the text expresses no positive or negative opinion about a mentioned target under a defined labeling policy. Conflating them damages both the user interface and evaluation.

Misconception: adding an evaluation script completes evaluation

Evaluation requires valid inputs, labels, splits and interpretation. The utility performs arithmetic; it cannot establish that your human labels or candidate policy are appropriate.

59. Code reading guide and ownership of each file

Read the core in this order: `engine.py`, `app.py`, `web/index.html`, `web/app.js`, then `web/style.css`. Start with one review and trace its text through every stage. The optional folder can remain closed until the first demo works.

File	Main responsibility	What to inspect first
engine.py	Rules and ranking	clean, analyze, recommend
app.py	Local API and optional JSON inference	analyze_live and POST routes
web/index.html	Page structure	Review inputs and result containers
web/app.js	Events, API, history rendering	Debounce and save handlers
web/style.css	Layout and visual design	Grid and mobile breakpoint
import_movielens.py	Small dataset conversion	CSV reading and rating averages
MovieStats.scala	Distributed batch statistics	Schemas, groupBy, join, writes
import_spark_catalog.py	Bounded export conversion	Part-file reading and JSON array
prepare_imdb.py	Shared review split	Hashes, groups, duplicate policy
train_baselines.py	BoW/TF-IDF comparison	Validation selection and JSON export
ngram_lab.py	Language-model experiment	Conditional probability and perplexity
word_vectors.py	CBOW/Skip-gram/GloVe	Shared parameters and neighbors
train_distilbert.py	Transformer experiment	Training loop and best checkpoint
train_aspects.py	Six supervised baselines	Leakage checks and per-aspect reports
corpus_lab.py	PMI and topics	Frequency threshold and LDA inputs
evaluate_pipeline.py	Metric calculations	Label, ranking, span and rubric functions

How to explain a function

State its input, transformations, output and failure boundary. For example, `analyze(text)` accepts English text, cleans it, finds opinion/aspect evidence and returns structured scores. It does not train anything, recognize every language or guarantee correct sarcasm analysis.

Source appendix

The following source appendix reproduces the runnable core, importers, Scala job and optional experiments. The ZIP remains the authoritative runnable file collection. Long source lines may wrap in the PDF; copy code from the extracted files rather than copying wrapped PDF text into your editor.

60. References, attribution and final research checklist

These primary sources provide dataset and API documentation. Project-specific design choices, worked examples, scope boundaries and evaluation plans in this handbook describe this delivery. Software APIs can change; preserve the versions used for a successful run.

R1. MovieLens-small README and GroupLens

https://files.grouplens.org/datasets/movielens/ml-latest-small-README.html

Dataset context citation: F. Maxwell Harper and Joseph A. Konstan, The MovieLens Datasets: History and Context, ACM TiiS, 2015, DOI 10.1145/2827872. Read the source usage and attribution conditions. Raw MovieLens data are not included in this ZIP.

R2. Stanford Large Movie Review Dataset

https://ai.stanford.edu/~amaas/data/sentiment/

Dataset paper: Andrew L. Maas and colleagues, Learning Word Vectors for Sentiment Analysis, ACL 2011. Download data separately and retain the provided source documentation. The new code does not inherit metrics from the uploaded project's trained model.

R3. scikit-learn TF-IDF documentation

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

Consult the parameters for smoothing, normalization and tokenization. The supplied training code exports the particular fitted values rather than recalculating IDF on live user text.

R4. Gensim Word2Vec documentation

https://radimrehurek.com/gensim/models/word2vec.html

Use the documented `sg`, `vector_size`, `window`, `min_count`, `workers` and training parameters. Save the experiment environment with your results.

R5. Stanford GloVe project

https://nlp.stanford.edu/projects/glove/

Paper: Jeffrey Pennington, Richard Socher and Christopher Manning, GloVe: Global Vectors for Word Representation, EMNLP 2014. Pretrained vector files are optional external inputs.

R6. Hugging Face Transformers model APIs

https://huggingface.co/docs/transformers/main_classes/model

The optional experiment uses `from_pretrained` and `save_pretrained`. Its chosen base model is `distilbert/distilbert-base-uncased`; record the exact downloaded revision for reproducibility.

R7. Apache Spark 3.5.7 quick start

https://spark.apache.org/docs/3.5.7/quick-start.html

The Scala package targets a specific compatible Spark/Scala combination. Use matching documentation and verify your installed environment before compilation.

Final research checklist

Can you identify which data trained each component? Can you reproduce the split? Can you distinguish a model probability from a heuristic index? Does every aspect label have human support? Are review IDs joined correctly? Are recommendations evaluated on unseen future interactions? Do summary claims point to evidence? Do the reported timings describe actual executed work? If these questions have clear answers, your project will be easier to defend and extend.

Appendix A. Complete selected source listings

These listings reproduce the runnable core, Scala job, experiments and tests. The extracted ZIP is authoritative because PDF lines can wrap visually.

How to read this appendix

Begin with engine.py and app.py, then trace one request through web/app.js. Read MovieStats.scala separately as the offline batch path. The optional scripts are experiments: they need downloaded data, extra packages and their own evaluation records before their outputs become project results.

Group	Files	Purpose
Live core	app.py, engine.py, web/*	Review analysis, API, history and ranking
Dataset paths	import_movielens.py, MovieStats.scala	Small import and distributed aggregation
NLP labs	optional/*	Training and evaluation experiments
Verification	tests/test_core.py	Deterministic behavioral checks

For source changes, edit the extracted files and rerun their tests. Do not copy visually wrapped lines from the PDF into an editor.

app.py

```
"""Run: python app.py. Local teaching server; no pip install required."""
import json
import math
import os
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import urlparse
from engine import analyze, recommend

ROOT = Path(__file__).resolve().parent
CATALOG_PATH = Path(os.environ.get('CINESCOPE_CATALOG', str(ROOT/'data/catalog.json')))
CATALOG = json.loads(CATALOG_PATH.read_text(encoding='utf-8'))
IDS = {m['id'] for m in CATALOG}
MODEL_PATH = ROOT/'data/sentiment.json'
MODEL = json.loads(MODEL_PATH.read_text()) if MODEL_PATH.exists() else None


def analyze_live(text):
    result = analyze(text)
    if MODEL and text.strip():
        from engine import clean, tokens
        from collections import Counter
        ts = tokens(clean(text))
        features = Counter(ts + [' '.join(ts[i:i+2]) for i in range(len(ts)-1)])
        pairs = [(i, features[t]*MODEL['idf'][i]) for t,i in MODEL['vocabulary'].items() if features[t]]
        norm = math.sqrt(sum(v*v for _,v in pairs)) or 1
        z = MODEL['intercept'] + sum(MODEL['weights'][i]*v/norm for i,v in pairs)
        p = 1/(1+math.exp(-max(-700,min(700,z))))
        result.update(score=round(100*p,1), label='positive' if p>=.5 else 'negative',
                      method='TF-IDF + Logistic Regression (overall only)')
    return result


class Handler(BaseHTTPRequestHandler):
    def reply(self, data, status=200):
        body = json.dumps(data,ensure_ascii=False).encode()
        self.send_response(status)
        self.send_header('Content-Type','application/json; charset=utf-8')
        self.send_header('Cache-Control','no-store')
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        path = urlparse(self.path).path
        if path == '/api/catalog':
            return self.reply({'movies': CATALOG, 'dataset': 'MovieLens' if CATALOG and CATALOG[0]['id'].
startswith('ml:') else 'authored demo',
                               'model': 'trained TF-IDF' if MODEL else 'rule-based baseline'})
        files = {'/':'index.html','/app.js':'app.js','/style.css':'style.css'}
        if path not in files:
            return self.reply({'error':'Not found'},404)
        content = (ROOT/'web'/files[path]).read_bytes()
        self.send_response(200)
        self.send_header('Content-Type',{'/':'text/html','/app.js':'text/javascript','/style.css':'text/
css'}[path]+'; charset=utf-8')
        self.end_headers()
        self.wfile.write(content)

    def do_POST(self):
        try:
            length = int(self.headers.get('Content-Length',0))
            if not 0 < length <= 200000:
                raise ValueError('Request must be 1-200000 bytes.')
            data = json.loads(self.rfile.read(length))
            if self.path == '/api/analyze':
                value = data.get('text','')
                if not isinstance(value,str) or len(value)>10000:
                    raise ValueError('Review must be text with at most 10000 characters.')
                return self.reply(analyze_live(value))
            if self.path == '/api/recommend':
                raw = data.get('history',[])
                if not isinstance(raw,list) or len(raw)>200:
                    raise ValueError('History must contain at most 200 films.')
                history = {}
                for h in raw:
                    rating = float(h['rating'])
                    review = h.get('review','')
                    if not math.isfinite(rating) or not .5<=rating<=5 or rating*2!=int(rating*2):
                        raise ValueError('Rating must be 0.5-5 in half-star steps.')
                    if not isinstance(review,str) or len(review)>10000:
                        raise ValueError('Invalid review.')
                    if h['id'] in IDS: history[h['id']] = dict(id=h['id'],rating=rating,review=review)
                return self.reply(recommend(CATALOG,list(history.values())))
```

```
            return self.reply({'error':'Not found'},404)
        except (ValueError, KeyError, TypeError, AttributeError):
            return self.reply({'error':'Invalid input. Check review length and rating.'},400)

if __name__ == '__main__':
    port = int(os.environ.get('PORT','8000'))
    print(f'CineScope: http://127.0.0.1:{port}',flush=True)
    ThreadingHTTPServer(('127.0.0.1',port),Handler).serve_forever()
```

engine.py

```
"""Small, explainable English baseline. Scores are heuristic indices, not accuracy."""
import html
import math
import re
from collections import Counter

ASPECTS = {
    'Story': ['story', 'plot', 'script', 'writing'],
    'Acting': ['acting', 'actor', 'actors', 'cast', 'performance', 'performances'],
    'Visuals': ['visuals', 'visual', 'cinematography', 'effects', 'animation'],
    'Music': ['music', 'soundtrack', 'score', 'songs'],
    'Pacing': ['pacing', 'pace', 'slow', 'dragged', 'rushed'],
    'Ending': ['ending', 'climax', 'finale'],
}
THEMES = {'Friendship': ['friendship', 'friends'], 'Family': ['family', 'parent', 'father', 'mother'],
          'Love': ['love', 'romance', 'romantic'], 'Survival': ['survival', 'survive'],
          'Revenge': ['revenge'], 'Identity': ['identity', 'memory'],
          'Space': ['space', 'planet', 'astronaut'], 'Justice': ['justice', 'crime']}
GENRES = {'Action': ['action', 'fight', 'chase'], 'Comedy': ['comedy', 'funny', 'humor'],
          'Drama': ['drama', 'dramatic'], 'Horror': ['horror', 'scary'],
          'Romance': ['romance', 'romantic'], 'Sci-Fi': ['science fiction', 'sci-fi', 'scifi'],
          'Thriller': ['thriller', 'suspense'], 'Animation': ['animation', 'animated']}
POS = set('good great excellent amazing brilliant wonderful beautiful enjoyable entertaining powerful
perfect fantastic loved love liked funny moving compelling stunning satisfying strong engaging fast'.
split())
NEG = set('bad awful terrible boring weak poor dull disappointing hated hate predictable confusing
rushed dragged ugly annoying slow worst pointless'.split())
NEGATORS = {'not', 'no', 'never', 'hardly', 'barely'}


def clean(text):
    text = html.unescape(text).replace('\u2019', "'")
    text = re.sub(r'<[^>]*>', ' ', text).lower()
    for a, b in {"can't": 'can not', "won't": 'will not', "n't": ' not',
                 '□': ' loved ', '□': ' good ', '□': ' awful ', '□': ' bad ',
                 'gr8': 'great', 'meh': 'dull', 'luv': 'love'}.items():
        text = text.replace(a, b) if a == "n't" or not a.isalpha() else re.sub(r'\b'+a+r'\b', b, text)
    return re.sub(r'\s+', ' ', text).strip()


def tokens(text):
    return re.findall(r"[a-z]+(?:'[a-z]+)?", text.lower())


def clauses(text):
    return [s.strip(' ,') for s in re.split(r'[.!?;]+|\b(?:but|however|although|yet)\b', text) if s.
strip(' ,')]


def mention(text, words):
    return any(re.search(r'(?<!\w)'+re.escape(w)+r'(?!\w)', text) for w in words)


def opinions(ts):
    result = []
    for i, t in enumerate(ts):
        sign = 1 if t in POS else -1 if t in NEG else 0
        if sign:
            if sum(x in NEGATORS for x in ts[max(0, i-3):i]) % 2:
                sign *= -1
            result.append((i, sign))
    return result


def index(values):
    return round(50 + 50 * sum(values)/len(values), 1) if values else None


def analyze(text):
    normalized = clean(text)
    parts = clauses(normalized)
    values = [v for p in parts for _, v in opinions(tokens(p))]
    score = index(values)
    result = {'cleaned': normalized, 'method': 'lexicon + nearest-aspect rules',
              'score': score, 'label': 'no evidence' if score is None else
              'positive' if score > 60 else 'negative' if score < 40 else 'mixed / balanced',
              'opinion_count': len(values), 'aspects': {}, 'themes': {}, 'genres': {}}
    collected = {a: [] for a in ASPECTS}
    for part in parts:
        ts = tokens(part)
        anchors = [(i, a) for i, word in enumerate(ts) for a, ws in ASPECTS.items() if word in ws]
        for position, value in opinions(ts):
            if not anchors:
```

```
                continue
            distance = min(abs(position-i) for i, _ in anchors)
            # Ties are skipped: avoid assigning one opinion to two nearby aspects.
            names = {a for i, a in anchors if abs(position-i) == distance}
            if distance <= 4 and len(names) == 1:
                collected[names.pop()].append((value, part))
    for name, evidence in collected.items():
        result['aspects'][name] = {'score': index([v for v, _ in evidence]),
                                   'evidence': list(dict.fromkeys(p for _, p in evidence))}
    for group, taxonomy in [('themes', THEMES), ('genres', GENRES)]:
        for name, ws in taxonomy.items():
            matched = [p for p in parts if mention(p, ws)]
            if matched:
                vals = [v for p in matched for _, v in opinions(tokens(p))]
                result[group][name] = {'score': index(vals), 'mentions': len(matched),
                    'coverage': round(100*len(matched)/max(1,len(parts)),1), 'evidence': matched}
    return result


def recommend(catalog, history, limit=6):
    """Rating-driven genre affinity + review-derived themes/aspect preferences."""
    by_id = {m['id']: m for m in catalog}
    seen = {h['id'] for h in history}
    genre, theme, aspect = Counter(), Counter(), Counter()
    for h in history:
        movie = by_id.get(h['id'])
        if not movie:
            continue
        weight = (h['rating']-3)/2
        for g in movie['genres']:
            genre[g] += weight
        review = analyze(h.get('review', ''))
        for t, r in review['themes'].items():
            if r['score'] is not None:
                theme[t] += (r['score']-50)/50
        for a, r in review['aspects'].items():
            if r['score'] is not None and r['score'] > 50:
                aspect[a] += (r['score']-50)/50
    scale = max(1, sum(abs((h['rating']-3)/2) for h in history))
    ranked = []
    for movie in catalog:
        if movie['id'] in seen:
            continue
        profiles = [analyze(r) for r in movie.get('reviews', [])]
        g = sum(genre[x] for x in movie['genres']) / (scale*max(1,len(movie['genres'])))
        present = {t for p in profiles for t in p['themes']}
        t = sum(theme[x] for x in present)/max(1,sum(abs(x) for x in theme.values()))
        matches = []
        for a, weight in aspect.items():
            scores = [p['aspects'][a]['score'] for p in profiles if p['aspects'][a]['score'] is not None]
            if scores:
                matches.append(weight * (sum(scores)/len(scores)-50)/50)
        a = sum(matches)/max(1,sum(aspect.values()))
        # Damped rating prior. Demo films have no fabricated community ratings.
        n, mean = movie.get('rating_count',0), movie.get('mean_rating',3)
        prior = ((n*mean + 20*3)/(n+20)-3)/2
        value = .65*g + .15*t + .15*a + .05*prior
        reasons = []
        liked = [x for x in movie['genres'] if genre[x] > 0]
        if liked: reasons.append('Liked genres: '+', '.join(liked))
        if t > 0: reasons.append('Review theme match')
        if a > 0: reasons.append('Review aspect match')
        if not reasons: reasons.append('Catalog fallback' if not history else 'Available unseen option')
        ranked.append({**movie, 'match': round(value,4), 'reasons': reasons,
                       'components': {'genre': round(g,3),'theme':round(t,3),'aspect':round(a,3),'prior':
round(prior,3)}})
    return sorted(ranked,key=lambda m:(-m['match'],m['title']))[:limit]
```

import_movielens.py

```
"""python import_movielens.py path/to/ml-latest-small [--reviews linked.csv]
Optional linked.csv: movieId,review. No attempt to join IMDb review file indexes.
"""
import argparse
import csv
import json
from collections import defaultdict
from pathlib import Path


def build(folder, reviews=None):
    sums, counts, texts = defaultdict(float), defaultdict(int), defaultdict(list)
    with (folder/'ratings.csv').open(encoding='utf-8-sig',newline='') as f:
        for row in csv.DictReader(f):
            value=float(row['rating']); mid=row['movieId']
            if .5<=value<=5:
                sums[mid]+=value; counts[mid]+=1
    if reviews:
        with reviews.open(encoding='utf-8-sig',newline='') as f:
            for row in csv.DictReader(f):
                text=row['review'].strip()
                if text and text not in texts[row['movieId']] and len(texts[row['movieId']])<20:
                    texts[row['movieId']].append(text[:10000])
    with (folder/'movies.csv').open(encoding='utf-8-sig',newline='') as f:
        return [dict(id='ml:'+r['movieId'],title=r['title'],
                     genres=[g for g in r['genres'].split('|') if g!='(no genres listed)'],
                     rating_count=counts[r['movieId']],mean_rating=sums[r['movieId']]/counts[r[
'movieId']] if counts[r['movieId']] else 3,
                     reviews=texts[r['movieId']],provenance='MovieLens metadata; optional user-supplied
linked reviews') for r in csv.DictReader(f)]

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('folder',type=Path);p.add_argument('--reviews',type=Path);
a=p.parse_args()
    destination=Path(__file__).resolve().parent/'data/movielens.json'
    destination.write_text(json.dumps(build(a.folder,a.reviews),ensure_ascii=False),encoding='utf-8')
    print('Wrote',destination,'\nSet CINESCOPE_CATALOG to this file, then restart app.py.')
```

web/index.html

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-
scale=1"><title>CineScope | Mid-sem studio</title><link rel="stylesheet" href="/style.css"></head>
<body><main><header><span class="eyebrow">CINESCOPE / MID-SEM STUDIO</span><h1>A review. A little
insight.<br>Your next movie.</h1><p>Explore the feeling behind the words, then build a watchlist around
your taste.</p><div id="mode" class="tag">Loading catalog...</div></header>
<div class="layout"><section class="panel"><div class="section-title"><h2>01 / Review lab</h2><span>LIVE
ANALYSIS</span></div>
<label for="movie">Movie</label><input id="search" placeholder="Search movie titles" aria-label="Search
movie titles"><select id="movie"></select><p id="metadata" class="muted"></p>
<label for="review">What did you think?</label><textarea id="review" maxlength="10000" placeholder="The
acting was excellent but the story was weak. The music was beautiful."></textarea>
<div class="row"><button id="example" class="secondary">Try an example</button><span id="status"
role="status" aria-live="polite">Type to analyze</span></div>
<div id="overall" class="overall">Your review analysis appears here.</div><h3>Craft & storytelling</
h3><p class="muted">Heuristic sentiment index: 0 negative, 50 balanced, 100 positive. Missing evidence
stays blank.</p><div id="aspects" class="scores"></div>
<h3>Themes & genre mentions</h3><p class="muted">These scores describe the matching clauses in your
review. They do not rate an entire genre.</p><div id="topics"></div>
<div class="save"><label for="rating">Your rating</label><select id="rating"><option>5</option><option>4.
5</option><option selected>4</option><option>3.5</option><option>3</option><option>2.5</
option><option>2</option><option>1.5</option><option>1</option><option>0.5</option></select><button
id="save">Save to my history</button></div></section>
<aside><section class="panel"><div class="section-title"><h2>02 / For you</h2><span>EXPLAINABLE PICKS</
span></div><p class="muted">Rate a few films to personalize these picks. Reviewed films are excluded.</
p><div id="recommendations"></div></section><section class="panel history"><h2>03 / Your history</h2><p
class="muted">Saved only in this browser. One local profile.</p><div id="history"></div><button
id="clear" class="secondary">Clear history</button></section></aside></div>
<footer>English teaching baseline. Sarcasm, implicit opinions and complex negation can be wrong. Demo
candidate reviews are synthetic. No accuracy claim is made.</footer></main><script src="/app.js"></
script></body></html>
```

web/app.js

```
const $ = id => document.getElementById(id);
const esc = s => String(s).replace(/[&<>"']/g, c => ({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;","'":
'&#39;'}[c]));
let catalog=[], history=[], key='', timer, request=0;
async function api(path, data) {
  const response = await fetch('/api/'+path, data === undefined ? {} : {method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify(data)});
  const result = await response.json();
  if (!response.ok) throw Error(result.error || 'Request failed');
  return result;
}
function options() {
  const selected=$('movie').value, q=$('search').value.toLowerCase();
  $('movie').innerHTML=catalog.filter(m=>m.title.toLowerCase().includes(q)).slice(0,200).map(m=>`<option
value="${esc(m.id)}">${esc(m.title)}</option>`).join('');
  if ([...$('movie').options].some(o=>o.value===selected)) $('movie').value=selected;
  metadata();
}
function metadata() {
  const m=catalog.find(m=>m.id===$('movie').value);
  $('metadata').textContent=m ? m.genres.join(' / ') + (m.rating_count ? ` • ${m.rating_count} dataset
ratings` : ' • Demo metadata') : 'No matching titles';
}
function scoreCard(name, r) {
  const score=r.score, color=score===null?'muted':score>60?'positive':score<40?'negative':'balanced';
  return `<div class="score"><div class="row"><strong>${esc(name)}</strong><b class="${color}">${
score===null?'No evidence':score+' / 100'}</b></div><div class="track"><div class="${color}"
style="width:${score===null?0:score}%"></div></div>${r.evidence.length?`<details><summary>Supporting
text</summary><p>${r.evidence.map(esc).join('<br>')}</p></details>`:''}</div>`;
}
async function analyze(version) {
  try {
    const r=await api('analyze',{text:$('review').value});
    if (version!==request) return;
    $('overall').innerHTML=`<span class="eyebrow">OVERALL SENTIMENT</span><div class="big">${r.
score===null?'—':r.score}<small>${r.score===null?'':' / 100'}</small></div><strong>${esc(r.label)}</
strong><p>${esc(r.method)}</p>`;
    $('aspects').innerHTML=Object.entries(r.aspects).map(([n,v])=>scoreCard(n,v)).join('');
    $('topics').innerHTML=['themes','genres'].map(group=>`<h4>${group==='themes'?'Themes':'Genre
mentions'}</h4>`+(Object.entries(r[group]).map(([n,v])=>scoreCard(n,v)+`<p class="muted">${v.coverage}%
clause coverage</p>`).join('')||'<p class="muted">No explicit mentions.</p>')).join('');
    $('status').textContent='Analysis ready';
  } catch(e) { if(version===request) $('status').textContent=e.message; }
}
async function refresh() {
  $('history').innerHTML=history.length?history.map(h=>`<p><strong>${esc(catalog.find(m=>m.id===h.id)?.
title||h.id)}</strong><br>${h.rating} / 5 <button class="remove secondary" data-id="${esc(h.
id)}">Remove</button></p>`).join(''):'<p class="muted">Your history is empty.</p>';
  try {
    const picks=await api('recommend',{history});
    $('recommendations').innerHTML=picks.map((m,i)=>`<article class="pick"><span class="number">${String(
i+1).padStart(2,'0')}</span><div><h3>${esc(m.title)}</h3><p>${m.genres.map(esc).join(' · ')}</p><small>${
m.reasons.map(esc).join(' · ')}</small><details><summary>Ranking details</summary><p>Genre ${m.
components.genre}; theme ${m.components.theme}; aspect ${m.components.aspect}; prior ${m.components.
prior}. Rank score ${m.match} (not a probability).</p></details></div></article>`).join('')||'<p>All
available titles are in your history.</p>';
  }catch(e){$('recommendations').textContent=e.message;}
}
function storeHistory(){localStorage.setItem(key,JSON.stringify(history));refresh();}
$('review').addEventListener('input',()=>{clearTimeout(timer);const version=++request;$('status').
textContent='Analyzing...';timer=setTimeout(()=>analyze(version),300);});
$('search').oninput=options;$('movie').onchange=metadata;
$('example').onclick=()=>{$('review').value='The acting was excellent but the story was weak. The music
was beautiful. I loved the friendship theme and the comedy was funny.';$('review').dispatchEvent(new
Event('input'));};
$('save').onclick=()=>{const id=$('movie').value;if(!id)return;history=history.filter(h=>h.id!==id);
history.push({id,rating:Number($('rating').value),review:$('review').value});history=history.slice(-200);
try{storeHistory();$('status').textContent='Saved to your history';}catch(e){$('status').
textContent='Browser storage unavailable: '+e.message;}};
$('clear').onclick=()=>{history=[];storeHistory();};
$('history').onclick=e=>{if(e.target.matches('.remove')){history=history.filter(h=>h.id!==e.target.
dataset.id);storeHistory();}};
(async()=>{try{const r=await api('catalog');catalog=r.movies;key='cinescope-midsem-'+r.dataset;try{
history=JSON.parse(localStorage.getItem(key)||'[]');if(!Array.isArray(history))history=[];}catch{
history=[];}history=history.filter(h=>catalog.some(m=>m.id===h.id));$('mode').textContent=r.dataset+'
catalog / '+r.model;options();await refresh();await analyze(++request);}catch(e){$('mode').textContent=e.
message;}})();
```

web/style.css

```
:root{color-scheme:dark;--bg:#101719;--panel:#192225;--line:#344044;--ink:#f0f0e6;--muted:#a5b1b0;--
accent:#c7e789}*{box-sizing:border-box}body{margin:0;background:var(--bg);color:var(--ink);font:15px/1.6
system-ui,sans-serif}main{max-width:1280px;margin:auto;padding:48px 32px}header{border-bottom:1px solid
var(--line);padding-bottom:34px;margin-bottom:32px}.eyebrow{font-size:11px;letter-spacing:2px;color:var(-
-accent);font-weight:700}h1{font-size:clamp(32px,4vw,52px);line-height:1.12;letter-spacing:-2px;margin:
20px 0}header p{color:var(--muted);max-width:630px}.tag{display:inline-block;border:1px solid var(--
line);padding:6px 12px;border-radius:30px;font-size:12px}.layout{display:grid;grid-template-columns:1.
12fr 1fr;gap:24px}.panel{background:var(--panel);border:1px solid var(--line);border-radius:16px;padding:
24px}.section-title,.row{display:flex;align-items:center;justify-content:space-between;gap:12px}.section-
title span{font-size:9px;letter-spacing:1px;color:var(--muted)}h2{font-size:18px;margin:0 0 16px}h3{font-
size:15px;margin:22px 0 8px}h4{margin:16px 0 8px}label{display:block;font-size:12px;font-weight:650;
margin:16px 0 6px}input,select,textarea{width:100%;background:#101719;border:1px solid #465357;border-
radius:8px;padding:11px;color:var(--ink);font:inherit}select{margin-top:8px}textarea{min-height:150px;
resize:vertical}button{cursor:pointer;background:var(--accent);color:#1b2b1a;border:0;border-radius:7px;
padding:10px 14px;font-weight:700}.secondary{background:transparent;color:var(--ink);border:1px solid
#536164;font-weight:400}button:hover{filter:brightness(1.12)}button:focus-visible,input:focus-visible,
textarea:focus-visible,select:focus-visible{outline:2px solid var(--accent);outline-offset:3px}.muted,
#status{color:var(--muted);font-size:12px}.overall{background:#243332;border-radius:10px;margin:22px 0;
padding:20px}.big{font-size:46px;line-height:1.3}.big small{font-size:16px;color:var(--muted)}.overall p{
margin:8px 0 0;font-size:11px;color:var(--muted)}.scores{display:grid;grid-template-columns:1fr 1fr;gap:
12px}.score{border:1px solid var(--line);padding:12px;border-radius:8px;font-size:12px}.track{height:4px;
background:#354245;border-radius:4px;margin-top:12px}.track div{height:4px;border-radius:4px}.positive{
color:#c7e789}.negative{color:#ef9b88}.balanced{color:#e3ca83}.track .positive{background:#c7e789}.track
.negative{background:#ef9b88}.track .balanced{background:#e3ca83}details{font-size:11px;color:var(--
muted);margin-top:10px}summary{cursor:pointer}details p{overflow-wrap:anywhere}.save{margin-top:22px;
display:grid;grid-template-columns:auto 80px 1fr;align-items:center;gap:12px}.save label,.save select{
margin:0}.pick{display:flex;gap:16px;border-bottom:1px solid var(--line);padding:18px 0}.pick:last-child{
border:0}.pick h3{margin:0;font-size:17px}.pick p{font-size:11px;margin:5px 0;color:var(--muted)}.pick
small{font-size:11px;color:var(--accent)}.number{font-size:12px;color:var(--muted);padding-top:4px}.
history{margin-top:24px}.remove{font-size:10px;padding:3px 7px;margin-left:8px}footer{font-size:11px;
color:var(--muted);margin-top:28px}@media(max-width:850px){.layout{grid-template-columns:1fr}main{
padding:28px 16px}.panel{padding:18px}.save{grid-template-columns:auto 70px}.save button{grid-column:1/-
1}}@media(max-width:400px){.scores{grid-template-columns:1fr}}
```

scala/src/main/scala/MovieStats.scala

```scala
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._

/** Usage: MovieStats <MovieLens folder> <new output folder>
  * Batch processing: no training and no Spark job per typed review.
  */
object MovieStats {
  def main(args: Array[String]): Unit = {
    require(args.length == 2, "Need MovieLens input folder and output folder")
    val spark = SparkSession.builder.appName("CineScope MovieLens Stats").getOrCreate()
    val ratingSchema = new StructType()
      .add("userId", IntegerType).add("movieId", IntegerType)
      .add("rating", DoubleType).add("timestamp", LongType)
    val movieSchema = new StructType()
      .add("movieId", IntegerType).add("title", StringType).add("genres", StringType)
    val ratings = spark.read.option("header", true).schema(ratingSchema)
      .csv(args(0) + "/ratings.csv")
      .na.drop(Seq("userId", "movieId", "rating", "timestamp"))
      .filter(col("rating").between(0.5, 5.0))
      .dropDuplicates("userId", "movieId", "rating", "timestamp")
    val movies = spark.read.option("header", true).schema(movieSchema)
      .csv(args(0) + "/movies.csv").na.drop(Seq("movieId", "title"))
      .dropDuplicates("movieId")
    val stats = ratings.groupBy("movieId").agg(
      count(lit(1)).as("rating_count"), avg("rating").as("mean_rating"))
    val joined = movies.join(stats, Seq("movieId"), "left")
      .na.fill(Map("rating_count" -> 0L, "mean_rating" -> 3.0))
    joined.write.mode("errorifexists").parquet(args(1) + "/movie_stats")
    joined.withColumn("genre", explode(split(col("genres"), "\\|")))
      .filter(col("genre") =!= "(no genres listed)")
      .groupBy("genre").agg(sum("rating_count").as("rating_events"),
        (sum(col("rating_count") * col("mean_rating")) / sum("rating_count")).as("mean_rating"))
      .write.mode("errorifexists").option("header", true).csv(args(1) + "/genre_stats")
    // A bounded export is for the small UI only. Full results stay distributed.
    joined.orderBy(desc("rating_count"), asc("movieId")).limit(2000)
      .select(concat(lit("ml:"),col("movieId")).as("id"), col("title"),
        split(col("genres"),"\\|").as("genres"), col("rating_count"), col("mean_rating"))
      .coalesce(1).write.mode("errorifexists").json(args(1) + "/ui_catalog")
    spark.stop()
  }
}
```

scala/import_spark_catalog.py

```
"""Convert Spark's bounded JSON-lines export to the app's JSON array."""
import json
import sys
from pathlib import Path
rows=[]
for file in Path(sys.argv[1]).glob('part-*.json'):
    for line in file.read_text(encoding='utf-8').splitlines():
        r=json.loads(line);r['reviews']=[];r['provenance']='MovieLens / Scala Spark batch'
        r['genres']=[g for g in r['genres'] if g!='(no genres listed)'];rows.append(r)
if not rows:raise ValueError('No part-*.json files found')
destination=Path(__file__).resolve().parents[1]/'data/movielens.json'
destination.write_text(json.dumps(rows,ensure_ascii=False),encoding='utf-8');print(destination)
```

optional/prepare_imdb.py

```
"""Convert extracted aclImdb into a fixed shared split: python optional/prepare_imdb.py PATH."""
import csv
import hashlib
import json
import re
import sys
from pathlib import Path
sys.path.insert(0,str(Path(__file__).resolve().parents[1]))
from engine import clean
ROOT=Path(__file__).resolve().parents[1]


def prepare(source, destination):
    buckets={}; conflicts=set(); total=0
    # Test seen first: retain official test membership when exact duplicates overlap.
    for split in ('test','train'):
        for label,name in [(0,'neg'),(1,'pos')]:
            urlfile=source/split/('urls_'+name+'.txt')
            urls=urlfile.read_text().splitlines() if urlfile.exists() else []
            for file in sorted((source/split/name).glob('*.txt')):
                total+=1; text=clean(file.read_text(encoding='utf-8'))
                fingerprint=hashlib.sha256(text.encode()).hexdigest()
                idx=int(file.name.split('_')[0]); match=re.search(r'tt\d+',urls[idx]) if idx<len(urls)
else None
                group=match.group() if match else fingerprint
                fold='test' if split=='test' else ('validation' if int(hashlib.sha256(group.encode()).
hexdigest()[:8],16)%5==0 else 'train')
                if fingerprint in buckets and buckets[fingerprint]['label']!=label: conflicts.add(
fingerprint)
                buckets.setdefault(fingerprint,dict(id=fingerprint,text=text,label=label,split=fold,
group=group))
    rows=[v for k,v in buckets.items() if k not in conflicts]
    if not rows: raise ValueError('No labeled files. Expected PATH/train/pos/*.txt etc.')
    with destination.open('w',newline='',encoding='utf-8') as f:
        w=csv.DictWriter(f,fieldnames=['id','text','label','split','group']);w.writeheader();w.writerows(
rows)
    report={'raw':total,'kept':len(rows),'conflicting_texts_removed':len(conflicts),
            'counts':{s:sum(r['split']==s for r in rows) for s in ('train','validation','test')},
            'policy':'Official test, exact dedup, training movie-group hash validation when URL metadata
exists. Test not grouped against train; inspect movie overlap separately.'}
    (destination.parent/'split_manifest.json').write_text(json.dumps(report,indent=2))
    print(report)

if __name__=='__main__':prepare(Path(sys.argv[1]),ROOT/'data/reviews.csv')
```

optional/train_baselines.py

```
"""Controlled BoW vs TF-IDF. Run after prepare_imdb.py; no test-driven selection."""
import csv
import json
import sys
from pathlib import Path
sys.path.insert(0,str(Path(__file__).resolve().parents[1]))
from engine import clean, tokens
ROOT=Path(__file__).resolve().parents[1]


def rows(path=ROOT/'data/reviews.csv'):
    with path.open(encoding='utf-8',newline='') as f:return list(csv.DictReader(f))


def run():
    from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import accuracy_score,f1_score,confusion_matrix
    from sklearn.pipeline import Pipeline
    data=rows();groups={s:[r for r in data if r['split']==s] for s in ('train','validation','test')}
    if any(not r for r in groups.values()):raise ValueError('Need nonempty train/validation/test splits.
')
    reports={}; winner=None
    for name,vectorizer in [('bow',CountVectorizer),('tfidf',TfidfVectorizer)]:
        candidates=[]
        for c in (.5,2.):
            model=Pipeline([('vector',vectorizer(preprocessor=clean,tokenizer=tokens,token_pattern=None,
lowercase=False,ngram_range=(1,2),max_features=30000,min_df=2)),('classifier',LogisticRegression(C=c,
max_iter=1000,random_state=42))])
            model.fit([r['text'] for r in groups['train']],[int(r['label']) for r in groups['train']])
            pred=model.predict([r['text'] for r in groups['validation']])
            candidates.append((f1_score([int(r['label']) for r in groups['validation']],pred,
average='macro'),c,model))
        validation,c,model=max(candidates,key=lambda x:x[0])
        test=groups['test']; gold=[int(r['label']) for r in test]; pred=model.predict([r['text'] for r
in test])
        reports[name]={'C':c,'validation_macro_f1':validation,'test_accuracy':accuracy_score(gold,pred),
'test_macro_f1':f1_score(gold,pred,average='macro'),'confusion_matrix_labels':[0,1],'confusion_matrix':
confusion_matrix(gold,pred,labels=[0,1]).tolist(),'test_n':len(test)}
        (ROOT/'data'/f'{name}_predictions.json').write_text(json.dumps([dict(id=r['id'],gold=y,pred=int(
p)) for r,y,p in zip(test,gold,pred)]))
        if name=='tfidf':winner=model
    v=winner.named_steps['vector'];lr=winner.named_steps['classifier']
    artifact={'vocabulary':v.vocabulary_,'idf':v.idf_.tolist(),'weights':lr.coef_[0].tolist(),
'intercept':float(lr.intercept_[0]),'classes':lr.classes_.tolist(),'model':'tfidf-logistic-regression',
'split':'data/split_manifest.json'}
    artifact['vocabulary']={k:int(v) for k,v in artifact['vocabulary'].items()}
    (ROOT/'data/sentiment.json').write_text(json.dumps(artifact))
    (ROOT/'data/baseline_metrics.json').write_text(json.dumps(reports,indent=2)); print(json.dumps(
reports,indent=2))

if __name__=='__main__':run()
```

optional/ngram_lab.py

```
"""A real count-based language model, separate from classifier n-gram features."""
import json
import math
from collections import Counter
from train_baselines import rows, ROOT, tokens, clean

class NGram:
    def __init__(self,n=2,alpha=1):self.n=n;self.alpha=alpha
    def fit(self,texts):
        sentences=[tokens(clean(t)) for t in texts]
        count=Counter(t for s in sentences for t in s)
        self.vocab={t for t,c in count.items() if c>=2}|{'<unk>','</s>'}
        self.count,self.context=Counter(),Counter()
        for s in sentences:
            seq=['<s>']*(self.n-1)+[t if t in self.vocab else '<unk>' for t in s]+['</s>']
            for i in range(self.n-1,len(seq)):
                context=tuple(seq[i-self.n+1:i]);self.count[context+(seq[i],)]+=1;self.context[
context]+=1
        return self
    def probability(self,context,word):
        context=tuple(t if t in self.vocab or t=='<s>' else '<unk>' for t in context)
        word=word if word in self.vocab else '<unk>'
        return (self.count[context+(word,)]+self.alpha)/(self.context[context]+self.alpha*len(self.
vocab))
    def perplexity(self,texts):
        loss=0;n=0
        for text in texts:
            seq=['<s>']*(self.n-1)+tokens(clean(text))+['</s>']
            for i in range(self.n-1,len(seq)):
                loss-=math.log(self.probability(seq[i-self.n+1:i],seq[i]));n+=1
        return math.exp(loss/n) if n else None
    def next_words(self,text,k=5):
        context=(['<s>']*(self.n-1)+tokens(clean(text)))[-(self.n-1):]
        return sorted(((w,self.probability(context,w)) for w in self.vocab),key=lambda x:(-x[1],x[0]))[:
k]

if __name__=='__main__':
    data=rows();train=[r['text'] for r in data if r['split']=='train'];test=[r['text'] for r in data if
r['split']=='test']
    output={}
    for n in (2,3):
        m=NGram(n).fit(train);output[str(n)]={'perplexity':m.perplexity(test),'after_the_movie':m.
next_words('the movie'),'vocab':len(m.vocab),'alpha':m.alpha}
    (ROOT/'data/ngram_metrics.json').write_text(json.dumps(output,indent=2));print(output)
```

optional/word_vectors.py

```
"""Train CBOW and Skip-gram on the shared training reviews; optional local GloVe txt."""
import argparse
import json
from train_baselines import rows, ROOT, tokens, clean

def run(glove=None):
    from gensim.models import Word2Vec, KeyedVectors
    sentences=[tokens(clean(r['text'])) for r in rows() if r['split']=='train']
    report={};queries=['story','acting','excellent','boring','music']
    for name,sg in [('cbow',0),('skipgram',1)]:
        m=Word2Vec(sentences=sentences,vector_size=100,window=5,min_count=5,sg=sg,workers=1,seed=42,
epochs=5)
        m.wv.save(str(ROOT/'data'/f'{name}.kv'))
        report[name]={q:m.wv.most_similar(q,topn=5) if q in m.wv else [] for q in queries}
    if glove:
        g=KeyedVectors.load_word2vec_format(glove,binary=False,no_header=True)
        report['glove']={q:g.most_similar(q,topn=5) if q in g else [] for q in queries}
    (ROOT/'data/word_vectors.json').write_text(json.dumps(report,indent=2));print(report)
if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--glove');a=p.parse_args();run(a.glove)
```

optional/train_distilbert.py

```
"""Optional GPU experiment. Same review IDs as the baseline. Not loaded by the demo."""
import argparse
import json
import random
from train_baselines import rows, ROOT

def run(epochs):
    import numpy as np
    import torch
    from transformers import AutoTokenizer, AutoModelForSequenceClassification
    from sklearn.metrics import f1_score,accuracy_score,confusion_matrix
    from torch.utils.data import DataLoader
    random.seed(42);np.random.seed(42);torch.manual_seed(42)
    device='cuda' if torch.cuda.is_available() else 'cpu'
    data=rows();splits={s:[r for r in data if r['split']==s] for s in ('train','validation','test')}
    tokenizer=AutoTokenizer.from_pretrained('distilbert/distilbert-base-uncased')
    model=AutoModelForSequenceClassification.from_pretrained('distilbert/distilbert-base-uncased',
num_labels=2).to(device)
    def collate(batch):
        encoded=tokenizer([r['text'] for r in batch],padding=True,truncation=True,max_length=256,
return_tensors='pt')
        encoded['labels']=torch.tensor([int(r['label']) for r in batch]);return {k:v.to(device) for k,v
in encoded.items()}
    loaders={s:DataLoader(r,batch_size=8,shuffle=s=='train',collate_fn=collate) for s,r in splits.items(
)}
    optimizer=torch.optim.AdamW(model.parameters(),lr=2e-5)
    def evaluate(loader):
        model.eval();gold=[];pred=[]
        with torch.no_grad():
            for batch in loader:
                gold.extend(batch['labels'].cpu().tolist());pred.extend(model(**batch).logits.argmax(-1).
cpu().tolist())
        return {'accuracy':accuracy_score(gold,pred),'macro_f1':f1_score(gold,pred,average='macro'),
'confusion_matrix':confusion_matrix(gold,pred,labels=[0,1]).tolist()},gold,pred
    best=-1;destination=ROOT/'data/distilbert'
    for epoch in range(epochs):
        model.train()
        for batch in loaders['train']:
            optimizer.zero_grad();loss=model(**batch).loss;loss.backward();torch.nn.utils.
clip_grad_norm_(model.parameters(),1.);optimizer.step()
        report,_,_=evaluate(loaders['validation']);print(epoch+1,report)
        if report['macro_f1']>best:
            best=report['macro_f1'];model.save_pretrained(destination);tokenizer.save_pretrained(
destination)
    model=AutoModelForSequenceClassification.from_pretrained(destination).to(device)
    report,gold,pred=evaluate(loaders['test']);report.update(validation_macro_f1=best,test_n=len(gold),
max_tokens=256,device=device,epochs=epochs)
    (ROOT/'data/distilbert_metrics.json').write_text(json.dumps(report,indent=2))
    (ROOT/'data/distilbert_predictions.json').write_text(json.dumps([dict(id=r['id'],gold=y,pred=p) for
r,y,p in zip(splits['test'],gold,pred)]));print(report)
if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--epochs',type=int,default=2);a=p.parse_args()
    if a.epochs<1:p.error('epochs must be positive')
    run(a.epochs)
```

optional/train_aspects.py

```
"""Human annotations only: review_id,text,aspect,label,split. Six small classifiers.
Labels: positive,negative,neutral,mixed,not_mentioned. No artificial gold labels.
This research script saves reports/models; the small frontend continues to use rules.
"""
import csv
import json
from pathlib import Path
from train_baselines import ROOT,clean,tokens

def run(path):
    import joblib
    from sklearn.pipeline import make_pipeline
    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import classification_report
    from engine import ASPECTS
    with open(path,encoding='utf-8',newline='') as f:data=list(csv.DictReader(f))
    labels=['positive','negative','neutral','mixed','not_mentioned'];seen={};result={}
    for r in data:
        if r['aspect'] not in ASPECTS or r['label'] not in labels or r['split'] not in ('train',
'validation','test'):raise ValueError('Invalid annotation value')
        for key in (r['review_id'],clean(r['text'])):
            if key in seen and seen[key]!=r['split']:raise ValueError('Review leaks across splits')
            seen[key]=r['split']
    for aspect in ASPECTS:
        train=[r for r in data if r['aspect']==aspect and r['split']=='train']
        test=[r for r in data if r['aspect']==aspect and r['split']=='test']
        if len({r['label'] for r in train})<2 or not test:
            result[aspect]={'status':'insufficient annotations'};continue
        model=make_pipeline(TfidfVectorizer(preprocessor=clean,tokenizer=tokens,token_pattern=None,
ngram_range=(1,2)),LogisticRegression(max_iter=1000,class_weight='balanced',random_state=42))
        model.fit([r['text'] for r in train],[r['label'] for r in train]);pred=model.predict([r['text']
for r in test])
        result[aspect]={'train_n':len(train),'test_n':len(test),'metrics':classification_report([r[
'label'] for r in test],pred,labels=labels,output_dict=True,zero_division=0)}
        joblib.dump(model,ROOT/'data'/f'aspect_{aspect.lower()}.joblib')
    (ROOT/'data/aspect_metrics.json').write_text(json.dumps(result,indent=2));print(result)
if __name__=='__main__':
    import sys
    run(sys.argv[1])
```

optional/corpus_lab.py

```
"""Real training corpus PMI and LDA. Run after IMDb preparation; no synthetic metrics."""
import json
import math
from collections import Counter
from train_baselines import ROOT,rows,clean,tokens

def run():
    from sklearn.feature_extraction.text import CountVectorizer
    from sklearn.decomposition import LatentDirichletAllocation
    texts=[r['text'] for r in rows() if r['split']=='train']
    bigrams=Counter();left=Counter();right=Counter()
    for text in texts:
        ts=tokens(clean(text))
        for a,b in zip(ts,ts[1:]):bigrams[a,b]+=1;left[a]+=1;right[b]+=1
    total=sum(bigrams.values())
    phrases=sorted([dict(phrase=a+' '+b,count=c,pmi=math.log2(c*total/(left[a]*right[b]))) for (a,b),c
in bigrams.items() if c>=10],key=lambda r:-r['pmi'])[:50]
    vector=CountVectorizer(max_features=10000,min_df=5,max_df=.9,stop_words='english')
    matrix=vector.fit_transform(texts);lda=LatentDirichletAllocation(n_components=6,max_iter=10,
random_state=42).fit(matrix)
    names=vector.get_feature_names_out();topics=[[names[i] for i in topic.argsort()[-10:][::-1]] for
topic in lda.components_]
    out={'pmi_min_count':10,'phrases':phrases,'topics_unlabeled':topics,'train_reviews':len(texts)}
    (ROOT/'data/corpus_lab.json').write_text(json.dumps(out,indent=2));print(out)
if __name__=='__main__':run()
```

optional/evaluate_pipeline.py

```
"""Score externally collected gold/predictions. No labels or results are fabricated.
Usage: python optional/evaluate_pipeline.py data/evaluation.json
Schema is in data/evaluation_template.json and the handbook.
"""
import json
import math
import sys
from collections import Counter

def label_metrics(rows):
    labels=sorted({r['gold'] for r in rows}|{r['pred'] for r in rows});per={};matrix=Counter((r['gold'],
r['pred']) for r in rows)
    for label in labels:
        tp=matrix[label,label];fp=sum(v for (g,p),v in matrix.items() if p==label and g!=label);fn=sum(v
for (g,p),v in matrix.items() if g==label and p!=label)
        precision=tp/(tp+fp) if tp+fp else 0;recall=tp/(tp+fn) if tp+fn else 0
        per[label]={'precision':precision,'recall':recall,'f1':2*precision*recall/(precision+recall) if
precision+recall else 0,'support':tp+fn}
    return {'n':len(rows),'accuracy':sum(r['gold']==r['pred'] for r in rows)/len(rows) if rows else None,
'macro_f1':sum(x['f1'] for x in per.values())/len(per) if per else None,'per_class':per,'labels':labels,
'confusion_matrix':[[matrix[g,p] for p in labels] for g in labels]}

def ranking(rows,k=10):
    scored=[]
    for r in rows:
        gold=set(r['relevant']);retrieved=list(dict.fromkeys(r['ranked']))[:k]
        if not gold:continue
        hits=[int(x in gold) for x in retrieved];dcg=sum(h/math.log2(i+2) for i,h in enumerate(hits));
ideal=sum(1/math.log2(i+2) for i in range(min(k,len(gold))))
        scored.append({'precision_at_k':sum(hits)/k,'recall_at_k':sum(hits)/len(gold),'ndcg_at_k':dcg/
ideal,'mrr':next((1/(i+1) for i,h in enumerate(hits) if h),0)})
    return {'total_queries':len(rows),'scored_queries':len(scored),'k':k,**{key:sum(r[key] for r in
scored)/len(scored) if scored else None for key in ['precision_at_k','recall_at_k','ndcg_at_k','mrr']}}

def evaluate(data):
    out={'sentiment':label_metrics(data.get('sentiment',[])),'aspects':{}}
    for name,rows in data.get('aspects',{}).items():out['aspects'][name]=label_metrics(rows)
    tp=fp=fn=0
    for row in data.get('ner',[]):
        gold={tuple(x) for x in row['gold']};pred={tuple(x) for x in row['pred']};tp+=len(gold&pred);
fp+=len(pred-gold);fn+=len(gold-pred)
    out['ner']={'tp':tp,'fp':fp,'fn':fn,'exact_span_type_f1':2*tp/(2*tp+fp+fn) if 2*tp+fp+fn else None}
    for task in ['retrieval','recommendations']:out[task]=ranking(data.get(task,[]),data.get('k',10))
    summaries=data.get('summaries',[])
    out['summaries']={'n':len(summaries)}
    for key in ['faithfulness','coverage','balance','readability']:
        vals=[r[key] for r in summaries if key in r]
        if any(not isinstance(v,(int,float)) or not math.isfinite(v) or not 1<=v<=5 for v in vals):raise
ValueError('Summary rubrics must be numeric 1-5')
        out['summaries'][key]={'n':len(vals),'mean':sum(vals)/len(vals) if vals else None}
    return out

if __name__=='__main__':
    from pathlib import Path
    path=Path(sys.argv[1]);output=path.with_name(path.stem+'_metrics.json')
    output.write_text(json.dumps(evaluate(json.loads(path.read_text())),indent=2));print(output)
```

tests/test_core.py

```
import json
import sys
import unittest
from pathlib import Path
sys.path.insert(0,str(Path(__file__).resolve().parents[1]))
from engine import analyze,clean,recommend
ROOT=Path(__file__).resolve().parents[1]

class CoreTests(unittest.TestCase):
    def test_opposite_aspects(self):
        r=analyze('The acting was excellent but the story was weak.')
        self.assertEqual(r['aspects']['Acting']['score'],100)
        self.assertEqual(r['aspects']['Story']['score'],0)
        self.assertIsNone(r['aspects']['Music']['score'])
    def test_negation(self):
        self.assertEqual(analyze("The acting wasn't good.")['score'],0)
        self.assertEqual(analyze('The story was not bad.')['score'],100)
    def test_no_evidence(self):
        self.assertIsNone(analyze('The story is about a person.')['aspects']['Story']['score'])
        self.assertIsNone(analyze('')['score'])
    def test_preprocess(self):
        self.assertEqual(clean('<p>gr8 &amp;amp; fun </p>'),'great & fun good')
    def test_theme_and_genre(self):
        r=analyze('The friendship story was excellent. The comedy was awful.')
        self.assertEqual(r['themes']['Friendship']['score'],100)
        self.assertEqual(r['genres']['Comedy']['score'],0)
        self.assertEqual(r['themes']['Friendship']['coverage'],50)
    def test_boundary(self):
        self.assertNotIn('Love',analyze('I loved it.')['themes'])
    def test_excludes_seen_and_prefers_liked_genre(self):
        movies=[dict(id='a',title='Seen',genres=['Comedy']),dict(id='b',title='Comedy',genres=[
'Comedy']),dict(id='c',title='Horror',genres=['Horror'])]
        r=recommend(movies,[dict(id='a',rating=5,review='')])
        self.assertEqual([m['id'] for m in r],['b','c'])
        d=recommend(movies,[dict(id='a',rating=1,review='')])
        self.assertEqual(d[0]['id'],'c')
    def test_review_signals_reach_ranking(self):
        movies=[dict(id='a',title='Seen',genres=[]),dict(id='b',title='Candidate',genres=[],reviews=[
'The friendship story was excellent.']),dict(id='c',title='Other',genres=[])]
        r=recommend(movies,[dict(id='a',rating=5,review='The friendship story was excellent.')])
        self.assertEqual(r[0]['id'],'b')
        self.assertGreater(r[0]['components']['theme'],0)
        self.assertGreater(r[0]['components']['aspect'],0)
    def test_ngram_normalizes(self):
        sys.path.insert(0,str(ROOT/'optional'))
        from ngram_lab import NGram
        m=NGram(2).fit(['the story is good','the story is bad'])
        self.assertAlmostEqual(sum(m.probability(['the'],w) for w in m.vocab),1)
        self.assertGreater(m.perplexity(['unseen text']),0)

if __name__=='__main__':unittest.main()
```